KI PRAXIS

Die Handbuchreihe für produktives Arbeiten mit künstlicher Intelligenz

BAND 10

Eigene KI-Agenten mit MCP

Vom Sprachmodell zum handlungsfähigen KI-System

Eugen Becker

1. Auflage, November 2026. Stand: 10. November 2026

Impressum und rechtlicher Hinweis

Reihe KI PRAXIS, Band 10: Eigene KI-Agenten mit MCP. Vom Sprachmodell zum handlungsfähigen KI-System.

Herausgeber und Autor: Eugen Becker.

1. Auflage, November 2026. Alle Rechte vorbehalten. Das Werk einschließlich aller Teile ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes bedarf der Zustimmung des Herausgebers.

Hinweis zu den Programmen: Alle Programme sind mit Python 3.12 und dem offiziellen MCP-Python-SDK in der Version 2.x ausgeführt worden. Die Ausgaben im Buch stammen aus diesen Läufen. Ollama wurde durch einen Emulator ersetzt, der feste Antworten liefert. Die Windows-Abschnitte sind nicht geprüft. Die Protokollangaben beziehen sich auf die Revision 2026-07-28 der Spezifikation.

Angaben zu Programmen, Oberflächen und Modellen sind versionsabhängig und können sich nach der Drucklegung ändern. Vor dem Einsatz ist die aktuelle Dokumentation des jeweiligen Projekts zu prüfen.

Inhaltsverzeichnis

Vorwort 9

Einleitung 10

Kapitel 1: Von der künstlichen Intelligenz zum KI-Agenten 11

1.1 Entwicklung moderner KI-Systeme 11
1.2 Was ist ein Large Language Model? 11
1.3 Wie Sprachmodelle Text erzeugen 12
1.4 Chatbots, Workflows und Agenten im Vergleich 12
1.5 Autonomie, Werkzeuge und Planung 13
1.6 Grenzen heutiger Agentensysteme 13
1.7 Praxisbeispiel: Chatbot und Agent bei derselben Aufgabe 13

Kapitel 2: Architektur moderner KI-Agenten 15

2.1 Grundaufbau eines Agenten 15
2.2 Das Sprachmodell als Verarbeitungseinheit 15
2.3 Der Agent Loop 15
2.4 Wahrnehmung, Kontext und Gedächtnis 16
2.5 Planung und Aufgabenzerlegung 16
2.6 Werkzeugaufrufe im Ablauf 16
2.7 Fehlerbehandlung und Orchestrierung 16

Kapitel 3: Das Model Context Protocol 18

3.1 Ziel und Entstehung von MCP 18
3.2 Warum standardisierte Schnittstellen 18
3.3 Klassische API-Anbindung und MCP im Vergleich 18
3.4 Rollen: Host, Client, Server 19
3.5 JSON-RPC: Nachrichten, Anfragen, Antworten, Benachrichtigungen 19
3.6 Das zustandslose Modell der Revision 2026-07-28 20
3.7 Das Modell der Revision 2025-11-25 zur Abgrenzung 20
3.8 Transporte: stdio und Streamable HTTP 21
3.9 Lokale und entfernte Server 21

Kapitel 4: Die drei zentralen MCP-Funktionen 22

4.1 Tools 22
4.2 Resources 23
4.3 Prompts 23
4.4 Zusammenspiel der drei Funktionen 23

Kapitel 5: Die Entwicklungsumgebung 25

5.1 Python-Versionen und Kompatibilität 25
5.2 Virtuelle Umgebungen 25
5.3 Installation des SDK 26
5.4 Visual Studio Code, Git und Projektstruktur 26
5.5 Debugging und Logging 27
5.6 Testumgebung unter Debian 27
5.7 Windows 11: Einrichtung 27

Kapitel 6: Eigene MCP-Server entwickeln 29

6.1 Aufbau eines Servers 29
6.2 Installation des SDK 29
6.3 Erster Server 29
6.4 Tools, Resources und Prompts implementieren 30
6.5 Projekt A: Systeminformationen-Server 30
6.6 Projekt B: Datei- und Verzeichnis-Server 35
6.8 Testen und Dokumentieren 42

Kapitel 7: Lokale Agenten mit Ollama 44

7.1 Lokale Modelle und ihre Grenzen 44
7.2 Architektur und Installation unter Debian 44
7.3 Modellwahl und Werkzeugfähigkeit 44
7.4 Die Chat-Schnittstelle und das Feld `tools` 45
7.5 Umwandlung von MCP-Werkzeugen in das Ollama-Format 45
7.6 Kontextfenster, Tokenverbrauch und Ressourcen 46
7.7 Offline-Betrieb 47
7.8 Integration: Ollama mit einem eigenen MCP-Server 47

Kapitel 8: Eigene KI-Agenten mit Python entwickeln 49

8.1 Aufbau eines Python-Agenten 49
8.2 Anbindung des Sprachmodells 49
8.3 MCP-Client einbinden 50
8.4 Werkzeugauswahl und Aufgabenplanung 51
8.5 Der Agent Loop 51
8.6 Mehrstufige Aufgaben und Konversationshistorie 53

8.7 Wiederholungen und Fehlerbehandlung 53
8.8 Tests des Agenten 53

Kapitel 9: KI-Agenten für Linux und Windows 54

9.1 Linux: Systeminformationen, Prozessor, Speicher, Festplatten . . . 54
9.2 Linux: Netzwerkstatus, Dienste, Systemprotokolle 54
9.3 Linux: Dateien und kontrollierte Befehle 55
9.4 Windows: PowerShell, Dienste, Ereignisprotokolle 55
9.5 Sichere Bereitstellung von Betriebssystemfunktionen 56

Kapitel 10: Datenbanken, APIs und externe Dienste 57

10.1 HTTP, JSON und REST 57
10.2 Authentifizierung und API-Schlüssel 57
10.3 SQLite als Gedächtnis- und Datenspeicher 57
10.4 PostgreSQL 58
10.5 MCP-Server für Datenbanken 58
10.6 Validierung, Timeouts und Rate Limiting 59
10.7 Projekt D: Datenbank- und API-Server 59

Kapitel 11: Gedächtnis und Wissensmanagement 66

11.1 Kurzzeitgedächtnis und Langzeitgedächtnis 66
11.2 Persistente Speicherung 66
11.3 Embeddings und Vektorsuche 66
11.4 Retrieval-Augmented Generation im Agenten 67
11.5 Grenzen und Risiken des Agentengedächtnisses 67
11.6 Projekt E: Persistentes Gedächtnissystem 67

Kapitel 12: Multi-Agent-Systeme 73

12.1 Grundlagen von Multi-Agent-Systemen 73
12.2 Spezialisierte Agenten 73
12.3 Rollen und Verantwortlichkeiten 74
12.4 Agentenkommunikation und Aufgabenverteilung 74
12.5 Koordination durch einen übergeordneten Agenten 75
12.6 Konflikte und Fehlererkennung 75
12.7 Grenzen der Zusammenarbeit 75
12.8 Projekt F: Multi-Agent-System mit Koordinator 76

Kapitel 13: Sicherheit und Zugriffskontrolle 81

13.1 Was Agenten zusätzlich gefährlich macht 81
13.2 Indirekte Prompt Injection bei Werkzeugen 81
13.3 Freigabe und Protokoll 82
13.4 Entfernte Server 82
13.5 Projekt G: Berechtigungsmodell 82

Kapitel 14: Monitoring, Betrieb und Deployment 88

14.1 Agenten überwachen 88
14.2 Logging 88
14.3 Tracing und Performance 89
14.4 Tokenverbrauch und Ressourcen 89
14.5 Verfügbarkeit 89
14.6 Docker 90
14.7 Linux-Dienste und Updates 90

14.8 Backup und Wiederherstellung 91
14.9 Versionsverwaltung und Deployment-Strategie 92

Kapitel 15: Praxisprojekt PERSONAL AI AGENT, MCP Edition 93

15.1 Anforderungen 93
15.2 Projektplanung 93
15.3 Architektur 93
15.4 Projektverzeichnis und Installation 94
15.5 bis 15.16 Etappen 94
15.4a Quellcode des Abschlussprojekts 94
15.6 Grenzen des Abschlussprojekts 102

Kapitel 16: Fehleranalyse und Troubleshooting 103

Kapitel 17: Die Zukunft autonomer KI-Agenten 105

17.1 Agentische KI 105
17.2 Lokale und hybride Agenten 105
17.3 Das MCP-Ökosystem und Interoperabilität 105
17.4 Agenten als digitale Mitarbeitende 105
17.5 Chancen und Risiken 106
17.6 Verfügbares und Zukünftiges 106

Glossar 107

Literatur- und Quellenverzeichnis 109

Stichwortverzeichnis 110

Vorwort

Ein Sprachmodell kann erklären, rechnen und formulieren. Was ihm fehlt, ist der Zugriff auf die Welt außerhalb des Gesprächs: auf Dateien, Systeminformationen, Datenbanken und Dienste. Ohne diesen Zugriff bleibt es ein sehr guter Gesprächspartner, der nichts tun kann. Mit dem Zugriff wird es zu einem System, das Aufgaben übernimmt.

Die Frage, wie ein Sprachmodell sicher und nachvollziehbar mit Werkzeugen arbeitet, hat sich in den letzten Jahren zu einem eigenen Fachgebiet entwickelt. Das Model Context Protocol, kurz MCP, ist ein offener Standard, der diese Verbindung regelt. Dieses Buch führt in den Standard ein und zeigt, wie man eigene Werkzeuge, eigene Server und schließlich einen eigenen Agenten entwickelt.

Die Reihe KI PRAXIS hat bisher gezeigt, wie man KI versteht, automatisiert, lokal betreibt und absichert. Dieser Band führt die Themen zusammen. Wer ihn liest, soll am Ende nicht nur wissen, was MCP ist, sondern ein funktionierendes System gebaut haben, das er versteht und betreiben kann.

Eugen Becker

Einleitung

Dieses Buch richtet sich an Menschen, die Python schreiben oder lernen und sich für KI-Systeme interessieren, die über reine Textantworten hinausgehen. Dazu gehören Fachinformatikerinnen und Fachinformatiker in der Ausbildung, IT-Administratoren, Entwicklerinnen, Homelab-Betreiber und alle, die ein lokales KI-System auf eigener Hardware aufbauen wollen.

Vorausgesetzt werden Grundkenntnisse in Python, im Umgang mit einem Terminal und im Betrieb eines Linux-Systems. Die Bände 3 und 4 der Reihe behandeln Automatisierung und lokale KI. Wo dieser Band auf ihre Inhalte aufbaut, verweist er darauf, statt sie zu wiederholen.

Die Kapitel 1 bis 5 bereiten das Fundament vor: die Begriffe, die Architektur, das Protokoll und die Entwicklungsumgebung. Die Kapitel 6 bis 8 bauen Server, Client und Agenten. Die Kapitel 9 bis 14 erweitern das System um Betriebssystemfunktionen, Datenbanken, Gedächtnis, mehrere Agenten, Sicherheit und Betrieb. Kapitel 15 führt alles in einem Abschlussprojekt zusammen. Kapitel 16 hilft bei Fehlern, Kapitel 17 blickt nach vorn.

Die Programme dieses Bandes sind vollständig abgedruckt und stehen als Archiv zum Herunterladen bereit. Sie wurden mit Python 3.12 und dem offiziellen MCP-Python-SDK in der Version 2.x ausgeführt. Wo ein Abschnitt nicht in der Referenzumgebung geprüft werden konnte, steht das ausdrücklich dabei.

Der technische Stand dieses Bandes ist der 10. November 2026. Das Protokoll und seine Werkzeuge entwickeln sich weiter. Die Versionsangaben im Buch sind deshalb die Grundlage für die Prüfung vor dem Einsatz.

Kapitel 1: Von der künstlichen Intelligenz zum KI-Agenten

1.1 Entwicklung moderner KI-Systeme

Sprachmodelle haben sich in wenigen Jahren von Forschungsprototypen zu Werkzeugen entwickelt, die im Alltag genutzt werden. Die ersten Anwendungen beschränkten sich auf Texte: Man stellte eine Frage und bekam eine Antwort. Die nächsten Schritte erweiterten diese Form um Bilder, Dokumente und Programmcode. Der aktuelle Schritt besteht darin, dass Modelle Aufgaben über mehrere Schritte hinweg erledigen und dabei auf externe Werkzeuge zugreifen.

1.2 Was ist ein Large Language Model?

Ein Large Language Model, kurz LLM, ist ein neuronales Netz, das mit sehr großen Textmengen trainiert wurde. Es lernt dabei, wie Sprache aufgebaut ist und welche Wörter in einem Zusammenhang wahrscheinlich folgen. Das Modell speichert kein Archiv von Dokumenten. Es besitzt Gewichte, also Zahlenwerte, die sein Verhalten bestimmen.

Für den Benutzer zählt vor allem eine Eigenschaft: Das Modell erzeugt Text, der plausibel klingt, aber nicht zwingend stimmt. Diese Eigenschaft heißt Halluzination und begleitet das ganze Buch. Ein Agent, der mit einem Modell arbeitet, braucht deshalb Werkzeuge, mit denen er Tatsachen prüfen kann, statt sich auf die Formulierung zu verlassen.

1.3 Wie Sprachmodelle Text erzeugen

Das Modell verarbeitet Text in kleinen Einheiten, den Token. Aus dem bisherigen Text berechnet es eine Wahrscheinlichkeit für jedes mögliche nächste Token und wählt eines aus. Dieser Vorgang wiederholt sich, bis eine Antwort entsteht.

Zwei Folgen sind für die Praxis wichtig. Erstens ist das Kontextfenster begrenzt: Das Modell sieht nur eine bestimmte Menge Text gleichzeitig. Zweitens entscheidet das Modell nur über den nächsten Text. Eine Handlung in der Außenwelt, etwa das Lesen einer Datei, führt nicht das Modell selbst aus. Das übernimmt ein Programm, das die Ausgabe des Modells interpretiert. Dieses Programm ist der Agent.

1.4 Chatbots, Workflows und Agenten im Vergleich

Die drei Begriffe werden oft durcheinandergebracht. Die Unterschiede betreffen vor allem die Frage, wer den nächsten Schritt bestimmt.

Ein Chatbot antwortet auf Nachrichten. Er führt keine Handlungen außerhalb des Gesprächs aus. Die Entscheidung über den nächsten Schritt liegt vollständig beim Menschen.

Ein Workflow ist eine fest vorgegebene Folge von Schritten. Jeder Schritt ist im Voraus festgelegt, das Programm entscheidet nicht selbst. Band 3 dieser Reihe behandelt Workflows ausführlich.

Ein Agent bestimmt die nächsten Schritte innerhalb eines Rahmens selbst. Er erhält ein Ziel, wählt Werkzeuge, wertet die Ergebnisse aus und entscheidet, ob er weitermacht. Die Entscheidung liegt bei einem Sprachmodell, der Rahmen bei dem Programmierer.

1.5 Autonomie, Werkzeuge und Planung

Autonomie bedeutet bei einem Agenten nicht Unabhängigkeit von jeder Kontrolle. Sie bedeutet, dass der Agent innerhalb definierter Grenzen selbst handelt. Die Grenzen legt der Mensch fest: welche Werkzeuge verfügbar sind, welche Aktionen eine Freigabe brauchen und wie viele Schritte erlaubt sind.

Werkzeuge sind Funktionen, die der Agent aufrufen kann. Ein Werkzeug kann die Systemauslastung abfragen, eine Datei lesen oder eine Datenbankabfrage ausführen. Das Modell kennt nur die Beschreibung des Werkzeugs und entscheidet, ob und wie es aufgerufen wird.

Planung bezeichnet die Zerlegung einer Aufgabe in Teilschritte. Ein Modell kann einen Plan im Text formulieren. Ob der Plan funktioniert, zeigt erst die Ausführung. Ein Agent muss deshalb nach jedem Schritt das Ergebnis prüfen und den Plan anpassen.

1.6 Grenzen heutiger Agentensysteme

Agenten machen Fehler, die sich nicht vollständig vorhersagen lassen. Sie wählen gelegentlich das falsche Werkzeug, übersehen Angaben im Ergebnis oder verlieren den Faden über viele Schritte. Sie sind außerdem anfällig für Anweisungen, die in fremden Inhalten versteckt sind. Kapitel 13 behandelt diese Gefahr.

Die wichtigste Konsequenz lautet: Ein Agent braucht Begrenzungen, die nicht vom Modell abhängen. Rechte, Pfadgrenzen, Freigaben und Protokolle wirken unabhängig davon, was das Modell vorschlägt.

1.7 Praxisbeispiel: Chatbot und Agent bei derselben Aufgabe

Die Aufgabe lautet: „Prüfe, ob auf dem Rechner noch genug Speicherplatz frei ist.“

Ein Chatbot antwortet vermutlich aus dem Gedächtnis: Er erklärt, wie man den Speicherplatz prüft, und bittet den Benutzer, den Befehl selbst auszuführen. Das Ergebnis hängt davon ab, ob der Benutzer den Befehl kennt und ausführt.

Ein Agent mit einem Werkzeug für den Speicherplatz ruft das Werkzeug auf, liest das Ergebnis und antwortet mit einer Bewertung. Der Wert im Beispiel ist frei gewählt: „Auf der Systempartition sind 42 Prozent belegt. Der Speicherplatz reicht aus.“ Er hat den Wert nicht geschätzt, sondern gemessen. Genau dieser Unterschied ist der Kern dieses Buches.

Quellen zu Kapitel 1:

Model Context Protocol, Spezifikation Revision 2026-07-28, Abschnitte zu Werkzeugen und Architektur, modelcontextprotocol.io.

Zu Halluzinationen bei Sprachmodellen siehe Band 1 und Band 2 der Reihe KI PRAXIS.

Kapitel 2: Architektur moderner KI-Agenten

2.1 Grundaufbau eines Agenten

Ein Agent besteht aus vier Teilen: dem Sprachmodell, dem Programm, das den Ablauf steuert, den Werkzeugen und dem Gedächtnis. Das Sprachmodell trifft Entscheidungen. Das Programm führt sie aus, prüft die Ergebnisse und entscheidet über den Abbruch. Die Werkzeuge liefern die Verbindung zur Außenwelt. Das Gedächtnis hält fest, was bisher geschehen ist.

2.2 Das Sprachmodell als Verarbeitungseinheit

Das Modell erhält bei jedem Schritt eine Nachricht mit allen bisherigen Eingaben, den Antworten und den Ergebnissen der Werkzeuge. Zusätzlich erhält es die Beschreibungen der verfügbaren Werkzeuge. Es antwortet entweder mit Text oder mit einem Werkzeugaufruf, also dem Namen eines Werkzeugs und den Argumenten.

2.3 Der Agent Loop

Der Kern jedes Agenten ist eine Schleife, der Agent Loop. Sie wiederholt drei Schritte: Das Modell erhält den aktuellen Stand und entscheidet. Wenn es ein Werkzeug aufruft, führt das Programm den Aufruf aus. Das Ergebnis geht zurück an das Modell. Die Schleife endet, wenn das Modell eine Antwort ohne Werkzeugaufruf gibt oder eine Grenze erreicht ist.

Die Schrittgrenze ist unverzichtbar. Ohne sie kann eine Schleife endlos laufen, etwa wenn ein Werkzeug immer denselben Fehler liefert. Band 3 hat dieses Muster bereits eingeführt. Hier wird es für MCP-Werkzeuge verwendet.

2.4 Wahrnehmung, Kontext und Gedächtnis

Ein Agent nimmt seine Umgebung über die Ergebnisse der Werkzeuge wahr. Der Kontext ist alles, was das Modell in einem Schritt sieht. Da das Kontextfenster begrenzt ist, muss das Programm entscheiden, welche Nachrichten es mitschickt. Kapitel 11 behandelt das Gedächtnis ausführlich.

2.5 Planung und Aufgabenzerlegung

Die Planung kann ausdrücklich erfolgen, indem das Modell zuerst einen Plan schreibt, oder implizit, indem es Schritt für Schritt entscheidet. Für wenig riskante Aufgaben genügt die schrittweise Entscheidung. Für Aufgaben mit Folgen lohnt sich ein Plan, den der Benutzer vor der Ausführung sieht.

2.6 Werkzeugaufrufe im Ablauf

Ein Werkzeugaufruf besteht aus dem Namen, den Argumenten und dem Ergebnis. Das Programm prüft die Argumente gegen das Schema des Werkzeugs, bevor es aufruft. Das Ergebnis wird als Nachricht an das Modell zurückgegeben. Kapitel 4 erklärt diese Schritte im Detail.

2.7 Fehlerbehandlung und Orchestrierung

Fehler können im Modell, im Werkzeug, im Transport oder im Ablauf entstehen. Ein robuster Agent unterscheidet zwischen einem erwarteten Fehler, etwa einer abgelehnten Pfadangabe, und einem unerwarteten Fehler, etwa einem Absturz. Erwartete Fehler gehen als Information an das Modell zurück. Unerwartete Fehler stoppen die Schleife und werden protokolliert.

Orchestrierung bezeichnet die Koordination mehrerer Komponenten oder Agenten. Kapitel 12 behandelt sie.

Quellen zu Kapitel 2:

Model Context Protocol, Spezifikation Revision 2026-07-28, Abschnitte zu Werkzeugaufrufen.

Ollama, Dokumentation der Chat-Schnittstelle mit Werkzeugen, ollama.com. Vor Verwendung auf den aktuellen Stand prüfen.

Kapitel 3: Das Model Context Protocol

3.1 Ziel und Entstehung von MCP

Das Model Context Protocol ist ein offener Standard, der beschreibt, wie KI-Anwendungen Werkzeuge, Datenquellen und Vorlagen von außen einbinden. Das Ziel ist, dass ein Werkzeug einmal entwickelt und von vielen Anwendungen genutzt werden kann. Ohne Standard müsste jeder Anbieter jede Anbindung neu programmieren.

Die Spezifikation wird versioniert. Die aktuelle Revision trägt das Datum 2026-07-28. Sie löste die Revision 2025-11-25 ab. Jede Revision hat ein eigenes Änderungsprotokoll, das die Unterschiede auflistet.

3.2 Warum standardisierte Schnittstellen

Ein Agent ohne Standard braucht für jeden Dienst einen eigenen Code. Mit einem Standard genügt eine Schnittstelle für alle Dienste, die sie anbieten. Der Client weiß dann, wie er Werkzeuge abfragt und aufruft, unabhängig davon, was das Werkzeug tut.

3.3 Klassische API-Anbindung und MCP im Vergleich

Eine klassische Anbindung ruft eine Programmierschnittstelle direkt auf. Der Entwickler kennt das Format der Anfrage, die Authentifizierung und die Antwort. Die Logik steckt im Programm, das die Schnittstelle nutzt.

Mit MCP beschreibt der Server seine Werkzeuge selbst. Der Client fragt sie ab und erhält Namen, Beschreibungen und Schemata. Das Sprachmodell kann daraus entscheiden, welches Werkzeug es braucht. Die Anbindung wird dadurch allgemeiner, die Verantwortung für die

Sicherheit bleibt aber beim Entwickler.

3.4 Rollen: Host, Client, Server

Die Spezifikation unterscheidet drei Rollen. Der Host ist die Anwendung, mit der der Benutzer arbeitet, etwa ein Entwicklungswerkzeug oder ein eigener Agent. Der Client ist ein Bestandteil des Hosts, der eine Verbindung zu einem Server hält. Der Server stellt Werkzeuge, Ressourcen und Prompts bereit.

Im Buch ist der eigene Agent gleichzeitig Host und Client. Die Server sind eigene Programme, die der Agent startet oder mit denen er sich verbindet.

3.5 JSON-RPC: Nachrichten, Anfragen, Antworten, Benachrichtigungen

MCP überträgt Nachrichten im Format JSON-RPC. Eine Anfrage enthält eine Methode, Parameter und eine Kennung. Die Antwort trägt dieselbe Kennung und entweder ein Ergebnis oder einen Fehler. Eine Benachrichtigung hat keine Kennung und erwartet keine Antwort.

Ein Werkzeugaufruf sieht im Kern so aus: Der Client sendet die Methode `tools/call` mit dem Namen des Werkzeugs und den Argumenten. Der Server antwortet mit dem Ergebnis, das aus Inhaltsblöcken besteht. Ein Fehler wird im Ergebnis markiert, damit das Modell ihn verarbeiten kann.

3.6 Das zustandslose Modell der Revision 2026-07-28

Die Revision 2026-07-28 hat das Protokoll zustandslos gemacht. Früher handelte der Client beim Verbindungsaufbau eine Sitzung mit dem Server aus. Er sendete die Methode `initialize`, erhielt die Fähigkeiten des Servers und bestätigte den Abschluss mit einer Benachrichtigung. Die Kennung der Sitzung begleitete jede weitere Anfrage.

Diese Sitzung entfällt in der neuen Revision. Jede Anfrage trägt ihre Protokollversion und die Fähigkeiten des Clients im Feld `_meta`. Wer die Fähigkeiten des Servers vorab kennen will, fragt sie mit `server/discover` ab. Ein Server kann dadurch hinter einem gewöhnlichen Lastverteiler laufen, ohne dass jede Anfrage bei derselben Instanz landen muss.

Für den Entwickler bedeutet das: Er muss keine Sitzungszustände im Server pflegen. Wer Zustände braucht, etwa ein Gedächtnis, speichert sie in der Anwendung oder in einer Datenbank. Kapitel 11 zeigt das.

3.7 Das Modell der Revision 2025-11-25 zur Abgrenzung

Ältere Clients und Server verwenden noch die Revision 2025-11-25. Sie beginnt mit dem Handshake aus `initialize` und `notifications/initialized`. Der Client nennt seine Fähigkeiten, der Server antwortet mit seinen eigenen. Die Sitzung gilt, bis sie geschlossen wird.

Das SDK in der Version 2.x unterstützt beide Modelle und handelt die Version beim Verbindungsaufbau aus. Dieses Buch verwendet das neue Modell. Das alte Modell wird hier erklärt, damit der Leser ältere Beispiele im Netz einordnen kann.

3.8 Transporte: stdio und Streamable HTTP

Ein Transport transportiert die Nachrichten zwischen Client und Server. Für lokale Server ist stdio der übliche Weg. Der Client startet den Server als Prozess und spricht über dessen Standardein- und -ausgabe mit ihm. Diese Verbindung ist für den Benutzer unsichtbar und braucht keinen offenen Port.

Für Server, die über ein Netz erreichbar sind, verwendet die Spezifikation Streamable HTTP. Der Server stellt einen HTTP-Endpunkt bereit, an den der Client Nachrichten sendet. Der frühere Transport HTTP mit Server-Sent Events gilt in der Revision 2026-07-28 als veraltet. Kapitel 13 behandelt den Schutz entfernter Server.

3.9 Lokale und entfernte Server

Ein lokaler Server läuft auf demselben Rechner wie der Client. Er hat Zugriff auf die Dateien und Dienste dieses Rechners, und genau darin liegt sein Nutzen und sein Risiko. Ein entfernter Server läuft auf einem anderen Rechner und ist über das Netz erreichbar. Er braucht Authentifizierung, Verschlüsselung und eine klare Abgrenzung der Rechte.

Quellen zu Kapitel 3:

Model Context Protocol, Spezifikation Revision 2026-07-28, modelcontextprotocol.io. Abschnitte zu Architektur, Nachrichten, Transporten und Versionierung.

Offizieller Blog des Model Context Protocol, Beitrag „The 2026-07-28 Specification“ vom 28. Juli 2026.

Offizieller Blog des Model Context Protocol, Beitrag „The 2026-07-28 MCP Specification Release Candidate“. Die Angaben zur Veraltung von Roots, Sampling und Logging stammen aus dem Release Candidate und sind in der endgültigen Spezifikation zu prüfen.

JSON-RPC 2.0 Specification, jsonrpc.org.

Kapitel 4: Die drei zentralen MCP-Funktionen

4.1 Tools

Ein Tool ist eine Funktion, die der Client über den Server aufrufen kann. Es besteht aus einem Namen, einer Beschreibung und einem Schema für die Eingabe. Die Beschreibung ist für das Modell wichtig, weil es sie liest, um zu entscheiden, ob es das Werkzeug braucht.

Das SDK erzeugt das Schema automatisch aus der Signatur der Python-Funktion und den Typangaben. Eine Funktion mit dem Parameter `pfad: str` erhält ein Schema, das einen Text verlangt. Ein Parameter mit Standardwert ist optional.

```
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("beispiel")


@mcp.tool()
def addiere(a: int, b: int) -> int:
    """Addiert zwei ganze Zahlen."""
    return a + b
```

Der Dekorator `@mcp.tool()` registriert die Funktion. Die Docstring-Zeile wird als Beschreibung übermittelt. Der Client erhält daraus einen Eintrag, den das Modell verwenden kann.

Das Rückgabeergebnis hat zwei Formen. Einfache Werte wie Zahlen oder Texte werden als Inhalt übertragen. Ein Rückgabewert mit einem festen Typ, etwa eine Datenklasse mit definierten Feldern, liefert zusätzlich ein strukturiertes Ergebnis. Die Projekte A und B verwenden Wörterbücher. Das SDK überträgt sie als JSON-Text, das strukturierte Ergebnis bleibt leer. Für die Prüfung auf Felder im Client ist das ausreichend, für eine maschinenlesbare Auswertung empfiehlt sich ein Rückgabetyp mit Schema.

Fehler behandelt der Server über die Ausnahme `ToolError`. Sie wird als Fehlerergebnis an den Client gesendet und vom Modell gelesen. Andere Ausnahmen gelten als unerwartete Abstürze. Die Projekte in Kapitel 6 unterscheiden beides konsequent.

4.2 Resources

Eine Resource ist eine Datenquelle, die der Client lesen kann. Sie hat eine Adresse, den URI, zum Beispiel `system://version`. Anders als ein Tool führt eine Resource keine Handlung aus. Sie liefert Inhalte, die die Anwendung als Kontext nutzen kann.

Statische Resources liefern immer denselben Inhalt. Dynamische Resources berechnen den Inhalt bei jedem Abruf, etwa den aktuellen Stand einer Datei. Eine Resource-Vorlage mit Platzhaltern im URI erlaubt, mehrere ähnliche Adressen mit einem Muster zu beschreiben.

4.3 Prompts

Ein Prompt ist eine Vorlage, die der Benutzer oder die Anwendung auswählt und mit Parametern füllt. Er liefert den Text, der an das Modell geht. Prompts sind nützlich, wenn eine Aufgabe immer ähnlich formuliert wird und die Formulierung gleich bleiben soll.

Ein Prompt im SDK ist eine Funktion, die einen Text zurückgibt. Ihre Parameter werden zu Parametern der Vorlage.

4.4 Zusammenspiel der drei Funktionen

Tools handeln oder fragen Daten ab, Resources liefern Hintergrund, Prompts strukturieren die Aufgabe. Ein Agent kann einen Prompt wählen, der ihn anweist, eine Resource zu lesen und dann ein Tool aufzurufen. Die Trennung hilft, Verantwortung zu verteilen: Tools tragen die Rechte, Resources die Daten, Prompts die Abläufe.

Quellen zu Kapitel 4:

Model Context Protocol, Spezifikation Revision 2026-07-28, Abschnitte zu Tools, Resources und Prompts.

MCP Python SDK, Version 2.x, Dokumentation zu MCPServer, Werkzeugen, Ressourcen und Prompts, github.com/modelcontextprotocol/python-sdk.

Kapitel 5: Die Entwicklungsumgebung

5.1 Python-Versionen und Kompatibilität

Das MCP-Python-SDK in der Version 2.x benötigt Python 3.10 oder neuer. Dieses Buch verwendet Python 3.12 als Referenz. Die Programme laufen auch unter 3.10 und 3.11. Sie nutzen keine Funktionen, die erst in einer neueren Version eingeführt wurden. Die Version lässt sich mit einem Befehl im Terminal prüfen:

```
python3 --version
```

Auf Debian Linux steht meist eine ältere Systemversion bereit. Wer eine neuere Version parallel installieren will, nutzt die Paketquellen der Distribution oder einen Versionsverwalter. Die Systemversion sollte für Entwicklungsarbeit nicht verändert werden, weil andere Werkzeuge auf sie angewiesen sind.

5.2 Virtuelle Umgebungen

Eine virtuelle Umgebung ist ein eigenes Verzeichnis mit einer eigenen Kopie des Python-Interpreters und eigenen Paketen. Sie verhindert, dass Projekte einander stören. Ein Projekt, das eine bestimmte Version des SDK braucht, bekommt diese Version in seiner Umgebung, unabhängig von anderen Projekten.

```
python3 -m venv .venv
source .venv/bin/activate
```

Nach der Aktivierung zeigt die Eingabeaufforderung den Namen der Umgebung. Die Umgebung wird mit `deactivate` verlassen. Das Verzeichnis `.venv` gehört nicht in die Versionsverwaltung, deshalb nimmt die Datei `.gitignore` es aus.

5.3 Installation des SDK

```
pip install "mcp[cli]" pytest
```

Das Paket `mcp` ist das offizielle SDK. Die Option `[cli]` installiert zusätzlich das Kommando `mcp`, mit dem man Server startet und testet. Für die Projekte dieses Buches genügt das Paket ohne diese Option, das Kommando wird aber für Experimente nützlich.

Für reproduzierbare Umgebungen legt man die Versionen in einer Datei fest. Das folgende Beispiel zeigt den Aufbau der Datei `requirements.txt` für das Projekt des Buches:

```
mcp==2.3.0
pytest==9.2.0
```

Die Installation erfolgt dann mit `pip install -r requirements.txt`. Ein Update des SDK ist eine bewusste Entscheidung, die man im Änderungsprotokoll nachliest, bevor man sie vornimmt.

5.4 Visual Studio Code, Git und Projektstruktur

Visual Studio Code eignet sich für die Entwicklung, weil es die virtuelle Umgebung erkennt und Tests in einer Seitenleiste ausführen kann. Die Erweiterung für Python gehört dazu. Die Auswahl des Interpreters erfolgt über die Befehlspalette, dort wählt man den Interpreter der Datei `.venv/bin/python`.

Git verfolgt Änderungen am Quellcode. Ein neues Projekt beginnt mit `git init`. Der Verlauf erlaubt es, eine funktionierende Fassung wiederherzustellen, wenn eine Änderung ein Werkzeug zerstört. Auf GitHub lassen sich Projekte veröffentlichen und mit anderen teilen.

Die Projektstruktur dieses Buches trennt Server, Tests und Hilfsprogramme. Jedes Projekt steht in einem eigenen Verzeichnis unterhalb von `projekte`. Die Tests liegen daneben, damit sie den Server importieren können, ohne ihn zu verändern.

5.5 Debugging und Logging

Ein MCP-Server schreibt seine Protokolle nicht in die Standardausgabe, weil diese für die Nachrichten des Protokolls genutzt wird. Ausgaben über `print` stören die Verbindung. Für Protokollmeldungen verwendet man das Modul `logging` mit einer Datei oder der Fehlerausgabe. Die Fehlerausgabe ist für den Client unsichtbar, der Benutzer sieht sie aber im Terminal, wenn er den Server direkt startet.

Für das Debugging eines Werkzeugs hilft es, den Server über die Testumgebung aufzurufen, statt ihn über den Client zu starten. Die Einzeltests in diesem Buch zeigen den Weg: Die Funktion wird direkt aufgerufen, und die Eingaben lassen sich festlegen.

5.6 Testumgebung unter Debian

Die Referenzumgebung ist Debian Linux. Für die Projekte sind keine Systemdienste nötig. Benötigt werden Python, das Modul `venv` und ein Editor. Auf Debian installiert man das Modul über das Paket `python3-venv`, falls es fehlt.

Der Test der Projekte läuft mit einem Befehl pro Projekt:

```
cd projekte/a_systeminfo
python -m pytest -q test_systeminfo.py
```

5.7 Windows 11: Einrichtung

Dieser Abschnitt ist nicht in der Referenzumgebung geprüft. Er beschreibt den üblichen Weg und ist als ungetestet zu lesen.

Unter Windows 11 installiert man Python aus dem offiziellen Installationsprogramm oder über den Paketmanager der Plattform. Bei der Installation sollte die Option zum Hinzufügen zum Pfad aktiviert sein. Eine virtuelle Umgebung erstellt man mit `py -m venv .venv`, aktiviert sie mit `.venv\Scripts\activate` und installiert die Pakete wie

unter Debian. Die Projekte aus Kapitel 6 lesen Dateien mit `/proc`. Diese Angaben gibt es unter Windows nicht. Projekt A ist deshalb nur unter Linux lauffähig. Die Windows-Werkzeuge in Kapitel 9 werden über PowerShell angebunden.

Quellen zu Kapitel 5:

Python-Dokumentation: venv, pip, logging. docs.python.org.

MCP Python SDK, Version 2.x, Installationsanleitung und Abhängigkeiten, github.com/modelcontextprotocol/python-sdk.

Paketseite mcp auf PyPI, Stand der Abfrage 10. November 2026.

Kapitel 6: Eigene MCP-Server entwickeln

6.1 Aufbau eines Servers

Ein Server ist ein Python-Programm mit einem Objekt `MCPServer`. An dieses Objekt werden Tools, Resources und Prompts angehängt. Der Aufruf `mcp.run()` startet den Server im Standardtransport stdio.

Jedes Projekt in diesem Kapitel besteht aus dem Server, einer Testdatei und einer Angabe der Abhängigkeiten. Die Tests laufen ohne Netzzugang und ohne ein Sprachmodell. Die Protokolltests starten den Server als Prozess und sprechen mit ihm, wie es ein Client tut.

6.2 Installation des SDK

```
python3 -m venv .venv
source .venv/bin/activate
pip install "mcp[cli]" pytest
```

Die virtuelle Umgebung trennt die Abhängigkeiten des Projekts vom System. Die Option `[cli]` bringt das Kommando `mcp` mit, das Server startet und testet. Die Tests in diesem Buch brauchen es nicht.

Getestet mit: Python 3.12, mcp 2.3.0, pytest 9.2.0. Die Werte stammen aus der Referenzumgebung dieses Buches.

6.3 Erster Server

Der Server aus dem Testlauf der Konzeption liefert die Systeminformationen. Er ist der Ausgangspunkt für Projekt A. Die vollständige Fassung steht im Abschnitt 6.5.

6.4 Tools, Resources und Prompts implementieren

Projekt A verwendet Tools für Speicher, Prozessor, Festplatte, Betriebszeit und Systeminformationen, eine Resource für die Versionsangabe und einen Prompt für die Zusammenfassung. Die Werte stammen aus den Dateien unter `/proc` und aus der Standardbibliothek. Das Projekt führt keine Befehle aus und ändert nichts im System. Es ist damit ein Beispiel für einen lesenden Server mit geringem Risiko.

Projekt B verwendet Tools für das Auflisten, Lesen, Durchsuchen und Schreiben von Dateien. Alle Pfade liegen innerhalb eines Arbeitsordners, den die Umgebungsvariable `MCP_BASIS` festlegt. Die Pfadprüfung löst relative und absolute Angaben auf und lehnt alles ab, was außerhalb liegt. Das Schreiben verlangt die ausdrückliche Angabe `bestaetigt=True`. Die Begrenzung stammt aus Band 8. Die Freigabe im Client folgt in Kapitel 13.

6.5 Projekt A: Systeminformationen-Server

Projektziel: Ein MCP-Server, der lesende Angaben zum Rechner bereitstellt und dabei keine Änderungen vornimmt.

Voraussetzungen: Debian Linux oder ein anderes System mit `/proc`, Python 3.10 oder neuer.

Verzeichnisstruktur:

```
projekte/a_systeminfo/
  systeminfo_server.py
  test_systeminfo.py
```

Quellcode des Servers:

Datei: projekte/a_systeminfo/systeminfo_server.py

```
"""Systeminformationen-Server (Projekt A, Kapitel 6).

Stellt nur lesende Werkzeuge bereit. Alle Angaben stammen aus
/proc und der Standardbibliothek.
```

```
"""
import os
import platform
import time
from pathlib import Path

from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError

mcp = MCPServer("systeminfo")
PROC = Path("/proc")


def lese_meminfo(pfad: Path = PROC / "meminfo") -> dict[str, int]:
    """Liest /proc/meminfo und gibt Werte in Kilobyte zurück."""
    werte: dict[str, int] = {}
    for zeile in pfad.read_text(encoding="utf-8").splitlines():
        name, _, rest = zeile.partition(":")
        teile = rest.split()
        if teile and teile[0].isdigit():
            werte[name] = int(teile[0])
    return werte


def lese_cpu_zeit(pfad: Path = PROC / "stat") -> tuple[int, int]:
    """Gibt (belegt, gesamt) Taktzeiten der CPU seit dem Systemstart
    zurück."""
    erste = pfad.read_text(encoding="utf-8").splitlines()[0].split()
    werte = [int(x) for x in erste[1:]]
    leerlauf = werte[3] + werte[4]  # idle und iowait
    gesamt = sum(werte)
    return gesamt - leerlauf, gesamt


def cpu_auslastung(pause: float = 0.2, pfad: Path = PROC / "stat") -> float:
    """Misst die CPU-Auslastung in Prozent über ein kurzes Intervall."""
    belegt1, gesamt1 = lese_cpu_zeit(pfad)
    time.sleep(pause)
    belegt2, gesamt2 = lese_cpu_zeit(pfad)
    differenz = gesamt2 - gesamt1
    if differenz <= 0:
        return 0.0
    return round(100.0 * (belegt2 - belegt1) / differenz, 1)


@mcp.tool()
def systeminfo() -> dict:
    """Gibt grundlegende Angaben zum Rechner zurück."""
    meminfo = lese_meminfo()
    return {
        "rechner": platform.node(),
        "betriebssystem": platform.system(),
        "kernel": platform.release(),
        "python": platform.python_version(),
        "kerne": os.cpu_count(),
```

```
        "speicher_gesamt_gb": round(meminfo["MemTotal"] / 1024 / 1024, 2),
    }


@mcp.tool()
def speicher() -> dict:
    """Gibt Arbeitsspeicher und Auslagerungsspeicher in Megabyte zurück."""
    m = lese_meminfo()
    gesamt = m["MemTotal"] // 1024
    verfuegbar = m["MemAvailable"] // 1024
    swap_gesamt = m.get("SwapTotal", 0) // 1024
    swap_frei = m.get("SwapFree", 0) // 1024
    return {
        "gesamt_mb": gesamt,
        "verfuegbar_mb": verfuegbar,
        "belegt_prozent": round(100.0 * (gesamt - verfuegbar) / gesamt, 1),
        "swap_gesamt_mb": swap_gesamt,
        "swap_belegt_mb": swap_gesamt - swap_frei,
    }


@mcp.tool()
def cpu() -> dict:
    """Gibt die CPU-Auslastung über ein kurzes Messintervall zurück."""
    return {"auslastung_prozent": cpu_auslastung(), "kerne": os.cpu_count()}


@mcp.tool()
def festplatte(pfad: str = "/") -> dict:
    """Gibt Speicherplatz eines Dateisystems in Gigabyte zurück."""
    ziel = Path(pfad).resolve()
    if not ziel.exists():
        raise ToolError(f"Pfad existiert nicht: {pfad}")
    st = os.statvfs(ziel)
    gesamt = st.f_blocks * st.f_frsize
    frei = st.f_bavail * st.f_frsize
    return {
        "pfad": str(ziel),
        "gesamt_gb": round(gesamt / 1024**3, 2),
        "frei_gb": round(frei / 1024**3, 2),
        "belegt_prozent": round(100.0 * (gesamt - frei) / gesamt, 1),
    }


@mcp.tool()
def betriebszeit() -> dict:
    """Gibt die Betriebszeit des Systems in Sekunden zurück."""
    sekunden = float((PROC /
    "uptime").read_text(encoding="utf-8").split()[0])
    return {"sekunden": int(sekunden), "tage": round(sekunden / 86400, 2)}


@mcp.resource("system://version")
def version() -> str:
    """Versionsangabe dieses Servers."""
```

```
    return "systeminfo 0.2.0"


@mcp.prompt()
def systemzusammenfassung(sprache: str = "de") -> str:
    """Vorlage für eine kurze Bewertung des Systemzustands."""
    return (f"Lies die Werte aus den Werkzeugen speicher, cpu und
    festplatte "
            f"und fasse den Zustand auf {sprache} in drei Sätzen zusammen.")


if __name__ == "__main__":
    mcp.run()
```

Quellcode der Tests:

Datei: projekte/a_systeminfo/test_systeminfo.py

```
from pathlib import Path

import pytest

import systeminfo_server as s
from mcp.server.mcpserver.exceptions import ToolError


def test_lese_meminfo_liest_werte(tmp_path: Path):
    datei = tmp_path / "meminfo"
    datei.write_text("MemTotal:       8000000 kB\nMemAvailable:   2000000
    kB\n",
                     encoding="utf-8")
    werte = s.lese_meminfo(datei)
    assert werte["MemTotal"] == 8000000
    assert werte["MemAvailable"] == 2000000


def test_cpu_auslastung_berechnet_prozent(monkeypatch):
    messungen = iter([(200, 1000), (300, 1100)])
    monkeypatch.setattr(s, "lese_cpu_zeit", lambda pfad=None:
    next(messungen))
    # Zwischen beiden Messungen: 100 von 100 Zeiteinheiten belegt
    assert s.cpu_auslastung(pause=0.0) == pytest.approx(100.0)


def test_cpu_auslastung_halb_belegt(monkeypatch):
    messungen = iter([(200, 1000), (250, 1100)])
    monkeypatch.setattr(s, "lese_cpu_zeit", lambda pfad=None:
    next(messungen))
    assert s.cpu_auslastung(pause=0.0) == pytest.approx(50.0)


def test_festplatte_meldet_unbekannten_pfad():
    with pytest.raises(ToolError):
        s.festplatte("/gibt/es/nicht/xyz")
```

```
def test_festplatte_liefert_felder():
    ergebnis = s.festplatte("/")
    assert set(ergebnis) == {"pfad", "gesamt_gb", "frei_gb",
    "belegt_prozent"}
    assert 0 <= ergebnis["belegt_prozent"] <= 100


def test_speicher_liefert_plausible_werte():
    ergebnis = s.speicher()
    assert ergebnis["gesamt_mb"] > 0
    assert 0 <= ergebnis["belegt_prozent"] <= 100


def test_betriebszeit_ist_positiv():
    assert s.betriebszeit()["sekunden"] > 0
```

Starten und prüfen:

```
cd projekte/a_systeminfo
source ../../.venv/bin/activate
python -m pytest -q test_systeminfo.py
```

Erwartetes Ergebnis: Alle sieben Tests bestehen. Ausgabe aus dem Testlauf in der Referenzumgebung: `7 passed`.

Erklärung: Die Funktion `lese_meminfo` nimmt einen Pfad als Parameter. Dadurch lässt sich der Test mit einer eigenen Datei durchführen, ohne auf den echten Zustand des Rechners angewiesen zu sein. Die CPU-Auslastung wird über zwei Messungen mit kurzer Pause berechnet, weil `/proc/stat` die Zeit seit dem Systemstart zählt. Der Test ersetzt die Messfunktion, damit das Ergebnis berechenbar ist.

Fehlerbehandlung: Ein unbekannter Pfad beim Festplatten-Werkzeug löst einen `ToolError` aus. Der Client erhält einen Fehler, den das Modell lesen kann, und der Server stürzt nicht ab.

Troubleshooting: Fehlt `/proc/meminfo`, läuft das Projekt auf einem Nicht-Linux-System. Das ist für dieses Projekt die einzige Voraussetzung, die nicht verhandelbar ist.

6.6 Projekt B: Datei- und Verzeichnis-Server

Projektziel: Ein MCP-Server, der Dateien in einem festen Arbeitsordner lesen, durchsuchen und mit Bestätigung schreiben kann.

Voraussetzungen: Python 3.10 oder neuer. Ein Arbeitsordner, den das Projekt über `MCP_BASIS` erhält. Ohne Angabe wird `./arbeitsordner` verwendet.

Verzeichnisstruktur:

```
projekte/b_dateien/
  dateien_server.py
  test_dateien.py
```

Quellcode des Servers:

Datei: projekte/b_dateien/dateien_server.py

```
"""Datei- und Verzeichnis-Server (Projekt B, Kapitel 6).

Alle Pfade werden innerhalb des Arbeitsordners MCP_BASIS aufgelöst.
Pfade außerhalb dieses Ordners werden abgelehnt, auch über ".." oder
absolute Pfade. Schreibende Werkzeuge verlangen die Bestätigung
bestaetigt=True.
"""
import os
from pathlib import Path

from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError

mcp = MCPServer("dateien")

MAX_ZEICHEN = 20000


def basis() -> Path:
    """Arbeitsordner aus der Umgebungsvariable MCP_BASIS, sonst
    ./arbeitsordner."""
    pfad = Path(os.environ.get("MCP_BASIS", "arbeitsordner"))
    return pfad.resolve()


def pfad_pruefen(name: str, basis_ordner: Path | None = None) -> Path:
    """Löst einen Pfad auf und prüft, ob er innerhalb des Arbeitsordners
    liegt."""
    wurzel = (basis_ordner or basis()).resolve()
    ziel = (wurzel / name).resolve()
    if ziel != wurzel and not ziel.is_relative_to(wurzel):
```

```
        raise ToolError(f"Pfad liegt außerhalb des Arbeitsordners: {name}")
    return ziel


@mcp.tool()
def liste_verzeichnis(pfad: str = ".") -> list[str]:
    """Listet die Einträge eines Verzeichnisses im Arbeitsordner auf."""
    ziel = pfad_pruefen(pfad)
    if not ziel.is_dir():
        raise ToolError(f"Kein Verzeichnis: {pfad}")
    return sorted(e.name + ("/" if e.is_dir() else "") for e in
    ziel.iterdir())


@mcp.tool()
def lese_datei(name: str, max_zeichen: int = 4000) -> str:
    """Liest eine Textdatei im Arbeitsordner. Die Länge ist begrenzt."""
    ziel = pfad_pruefen(name)
    if not ziel.is_file():
        raise ToolError(f"Keine Datei: {name}")
    grenze = min(max(max_zeichen, 1), MAX_ZEICHEN)
    text = ziel.read_text(encoding="utf-8", errors="replace")
    return text[:grenze]


@mcp.tool()
def suche_text(muster: str, name: str = ".") -> list[dict]:
    """Sucht ein Textmuster in allen Dateien unterhalb eines Pfads."""
    wurzel = pfad_pruefen(name)
    treffer: list[dict] = []
    for datei in sorted(wurzel.rglob("*")):
        if not datei.is_file():
            continue
        try:
            zeilen = datei.read_text(encoding="utf-8").splitlines()
        except UnicodeDecodeError:
            continue
        for nummer, zeile in enumerate(zeilen, 1):
            if muster in zeile:
                treffer.append({
                    "datei": str(datei.relative_to(basis())),
                    "zeile": nummer,
                    "text": zeile.strip()[:200],
                })
    return treffer[:200]


@mcp.tool()
def schreibe_datei(name: str, inhalt: str, bestaetigt: bool = False) -> str:
    """Schreibt eine Textdatei im Arbeitsordner. Ohne Bestätigung wird
    nichts geschrieben."""
    if not bestaetigt:
        raise ToolError("Schreibzugriff ohne Bestätigung abgelehnt
    (bestaetigt=True nötig)")
    ziel = pfad_pruefen(name)
```

```
    ziel.parent.mkdir(parents=True, exist_ok=True)
    ziel.write_text(inhalt, encoding="utf-8")
    return f"geschrieben: {name} ({len(inhalt)} Zeichen)"


if __name__ == "__main__":
    mcp.run()
```

Quellcode der Tests:

Datei: projekte/b_dateien/test_dateien.py

```
from pathlib import Path

import pytest

import dateien_server as d
from mcp.server.mcpserver.exceptions import ToolError


@pytest.fixture
def wurzel(tmp_path: Path) -> Path:
    (tmp_path / "notizen").mkdir()
    (tmp_path / "notizen" / "a.txt").write_text("Hallo Welt\nZeile
    zwei\n", encoding="utf-8")
    return tmp_path


def test_pfad_innerhalb_wird_akzeptiert(wurzel):
    ziel = d.pfad_pruefen("notizen/a.txt", wurzel)
    assert ziel == (wurzel / "notizen" / "a.txt").resolve()


@pytest.mark.parametrize("name", ["../geheim.txt",
    "notizen/../../geheim.txt", "/etc/passwd"])
def test_pfad_ausserhalb_wird_abgelehnt(wurzel, name):
    with pytest.raises(ToolError):
        d.pfad_pruefen(name, wurzel)


def test_liste_verzeichnis(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    assert d.liste_verzeichnis(".") == ["notizen/"]


def test_lese_datei_begrenzt(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    assert d.lese_datei("notizen/a.txt", max_zeichen=5) == "Hallo"


def test_suche_text_findet_zeile(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    treffer = d.suche_text("Zeile")
    assert treffer == [{"datei": "notizen/a.txt", "zeile": 2, "text":
```

```
    "Zeile zwei"}]


def test_schreiben_ohne_bestaetigung_wird_abgelehnt(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    with pytest.raises(ToolError):
        d.schreibe_datei("neu.txt", "x")
    assert not (wurzel / "neu.txt").exists()


def test_schreiben_mit_bestaetigung(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    d.schreibe_datei("neu.txt", "Inhalt", bestaetigt=True)
    assert (wurzel / "neu.txt").read_text(encoding="utf-8") == "Inhalt"


def test_schreiben_ausserhalb_wird_abgelehnt(monkeypatch, wurzel):
    monkeypatch.setenv("MCP_BASIS", str(wurzel))
    with pytest.raises(ToolError):
        d.schreibe_datei("../x.txt", "x", bestaetigt=True)
```

Starten und prüfen:

```
cd projekte/b_dateien
source ../../.venv/bin/activate
python -m pytest -q test_dateien.py
```

Erwartetes Ergebnis: Alle zehn Tests bestehen. Ausgabe aus dem Testlauf: `10 passed`.

Erklärung: Die Funktion `pfad_pruefen` löst den Pfad mit `resolve()` auf. Damit werden `..`-Bestandteile und Verknüpfungen aufgelöst, bevor die Prüfung erfolgt. Die Prüfung `is_relative_to` entscheidet, ob das Ziel innerhalb des Arbeitsordners liegt. Der Test prüft dazu absolute Pfade und Pfade mit `..`.

Schreiben: Das Werkzeug `schreibe_datei` verlangt `bestaetigt=True`. Der Standardwert ist `False`. Damit führt ein Aufruf ohne ausdrückliche Angabe nichts aus. Das Modell muss die Bestätigung bewusst setzen, und der Client kann sie über die Freigabe kontrollieren. Die Freigabe im Client folgt in Kapitel 13.

Protokolltest: Die Datei `tests/test_protokoll.py` startet beide Server als Prozesse und prüft die Werkzeuge über das Protokoll. Ein Pfad

außerhalb des Arbeitsordners wird als Fehler gemeldet. Ein Schreibaufruf ohne Bestätigung bleibt wirkungslos. Ausgabe aus dem Testlauf: `ALLE PROTOKOLLTESTS BESTANDEN`.

Projektziel: Ein MCP-Server, der Namensauflösung, Portprüfung und Erreichbarkeit diagnostiziert. Er verändert keine Einstellungen.

Voraussetzungen: Python 3.10 oder neuer. Das Programm `ping` ist optional. Ohne `ping` liefert das Werkzeug `ping` eine verständliche Fehlermeldung, die übrigen Werkzeuge funktionieren weiter.

Verzeichnisstruktur:

```
projekte/c_netzwerk/
  netzwerk_server.py
  test_netzwerk.py
```

Quellcode des Servers:

Datei: projekte/c_netzwerk/netzwerk_server.py

```
"""Netzwerkdiagnose-Server (Projekt C, Kapitel 6).

Stellt nur Diagnosen bereit: Namensauflösung, Portprüfung und
Erreichbarkeit. Es werden keine Netzwerkeinstellungen verändert.
Systembefehle werden ohne Shell mit fester Argumentliste gestartet,
und Eingaben werden vorher geprüft.
"""
import re
import shutil
import socket
import subprocess

from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError

mcp = MCPServer("netzwerk")

HOST_MUSTER = re.compile(r"^[A-Za-z0-9]([A-Za-z0-9\-\.]{0,251}[A-Za-z0-9])
    ?$")
TIMEOUT = 3.0


def host_pruefen(host: str) -> str:
    """Lässt nur einfache Hostnamen und IP-Adressen zu."""
    if not HOST_MUSTER.fullmatch(host):
        raise ToolError(f"Ungültiger Hostname: {host!r}")
    return host
```

```
def port_pruefen(port: int) -> int:
    if not 1 <= port <= 65535:
        raise ToolError(f"Port außerhalb des gültigen Bereichs: {port}")
    return port


@mcp.tool()
def dns_aufloesung(host: str) -> dict:
    """Löst einen Hostnamen in IP-Adressen auf."""
    host_pruefen(host)
    try:
        eintraege = socket.getaddrinfo(host, None, proto=socket.IPPROTO_TCP)
    except socket.gaierror as fehler:
        raise ToolError(f"Namensauflösung fehlgeschlagen: {host}
    ({fehler.strerror})")
    adressen = sorted({e[4][0] for e in eintraege})
    return {"host": host, "adressen": adressen}


@mcp.tool()
def port_offen(host: str, port: int) -> dict:
    """Prüft, ob ein TCP-Port auf einem Host Verbindungen annimmt."""
    host_pruefen(host)
    port_pruefen(port)
    try:
        with socket.create_connection((host, port), timeout=TIMEOUT):
            offen = True
    except OSError:
        offen = False
    return {"host": host, "port": port, "offen": offen}


@mcp.tool()
def ping(host: str, anzahl: int = 3) -> dict:
    """Sendet ICMP-Echoanfragen, wenn das Programm ping vorhanden ist."""
    host_pruefen(host)
    if not 1 <= anzahl <= 10:
        raise ToolError("Anzahl muss zwischen 1 und 10 liegen")
    programm = shutil.which("ping")
    if programm is None:
        raise ToolError("ping ist auf diesem System nicht installiert")
    # Argumentliste ohne Shell: Der Hostname kann keine Befehle
    einschleusen.
    lauf = subprocess.run([programm, "-c", str(anzahl), "-W", "2", host],
                          capture_output=True, text=True, timeout=anzahl
    * 3 + 5,
                          check=False)
    return {"host": host, "erfolgreich": lauf.returncode == 0,
            "ausgabe": lauf.stdout[-800:]}


if __name__ == "__main__":
    mcp.run()
```

Quellcode der Tests:

Datei: projekte/c_netzwerk/test_netzwerk.py

```
import pytest

import netzwerk_server as n
from mcp.server.mcpserver.exceptions import ToolError


@pytest.mark.parametrize("host", ["localhost", "127.0.0.1",
    "beispiel-host.lan"])
def test_gueltige_hosts(host):
    assert n.host_pruefen(host) == host


@pytest.mark.parametrize("host", ["", "a;rm -rf /", "$(id)", "a b", "-x",
    "a" * 300])
def test_ungueltige_hosts(host):
    with pytest.raises(ToolError):
        n.host_pruefen(host)


@pytest.mark.parametrize("port", [0, 65536, -1])
def test_ungueltige_ports(port):
    with pytest.raises(ToolError):
        n.port_pruefen(port)


def test_dns_localhost():
    ergebnis = n.dns_aufloesung("localhost")
    assert "127.0.0.1" in ergebnis["adressen"] or "::1" in
    ergebnis["adressen"]


def test_dns_unbekannter_host_meldet_fehler():
    with pytest.raises(ToolError):
        n.dns_aufloesung("gibt-es-nicht.invalid")


def test_port_geschlossen_auf_localhost():
    # Port 1 ist auf Standardsystemen nicht belegt.
    ergebnis = n.port_offen("127.0.0.1", 1)
    assert ergebnis == {"host": "127.0.0.1", "port": 1, "offen": False}


def test_port_offen_mit_eigenem_listener():
    import socket
    server = socket.socket()
    server.bind(("127.0.0.1", 0))
    server.listen(1)
    port = server.getsockname()[1]
    try:
        assert n.port_offen("127.0.0.1", port)["offen"] is True
```

```
    finally:
        server.close()


def test_ping_ohne_programm_meldet_fehler(monkeypatch):
    monkeypatch.setattr(n.shutil, "which", lambda name: None)
    with pytest.raises(ToolError):
        n.ping("localhost")


def test_ping_anzahl_wird_geprueft():
    with pytest.raises(ToolError):
        n.ping("localhost", anzahl=0)
```

Starten und prüfen:

```
cd projekte/c_netzwerk
source ../../.venv/bin/activate
python -m pytest -q test_netzwerk.py
```

Erwartetes Ergebnis: `18 passed`.

Erklärung: Die Prüfung der Hostnamen erfolgt mit einem regulären Ausdruck, der nur Buchstaben, Ziffern, Bindestriche und Punkte zulässt. Shell-Zeichen wie `;` oder `$(...)` werden abgelehnt. Das Werkzeug `ping` ruft das Programm nicht über eine Shell auf, sondern mit einer festen Argumentliste. Die Eingabe kann so keine zusätzlichen Befehle einschleusen. Die Tests prüfen gültige und ungültige Namen einzeln.

Sicherheit: Die Portprüfung öffnet eine echte Verbindung. Das Werkzeug kann deshalb Netzwerkverkehr auslösen. Auf fremden Systemen ist das nur mit Erlaubnis zulässig. Der Server prüft deshalb standardmäßig nur die Angaben, die der Benutzer ausdrücklich nennt.

6.8 Testen und Dokumentieren

Ein Server gilt als fertig, wenn drei Bedingungen erfüllt sind. Die Einzeltests prüfen die Logik. Der Protokolltest prüft das Zusammenspiel mit einem Client. Die Dokumentation beschreibt Werkzeuge, Parameter, Rechte und Fehler, damit ein Benutzer ohne den Quellcode auskommt.

Quellen zu Kapitel 6:

MCP Python SDK, Version 2.x, github.com/modelcontextprotocol/python-sdk, Release v2.0.0 vom 28. Juli 2026.

Paketseite mcp auf PyPI, Stand der Abfrage 10. November 2026.

Python-Dokumentation: pathlib, json, unittest.mock. docs.python.org.

Pytest-Dokumentation, docs.pytest.org.

Kapitel 7: Lokale Agenten mit Ollama

7.1 Lokale Modelle und ihre Grenzen

Ein lokales Sprachmodell läuft auf dem eigenen Rechner. Die Daten verlassen das Gerät nicht, und die Nutzung kostet keine Gebühr pro Anfrage. Die Grenzen liegen in der Leistung. Kleinere Modelle antworten schneller, verstehen aber komplexe Aufgaben schlechter. Größere Modelle brauchen viel Speicher und sind langsamer. Jede Modellwahl ist daher ein Kompromiss aus Qualität, Geschwindigkeit und Ressourcen.

7.2 Architektur und Installation unter Debian

Ollama ist ein Inferenzserver. Er lädt Modelle, stellt sie über eine HTTP-Schnittstelle bereit und führt die Berechnungen aus. Die Installation und die Verwaltung der Modelle sind in Band 4 der Reihe ausführlich beschrieben. Dieses Kapitel setzt eine laufende Installation voraus.

Ob Ollama erreichbar ist, prüft man mit einer Anfrage an die Modellliste. Die Adresse lautet standardmäßig `http://127.0.0.1:11434`. Eine Antwort mit einer Liste von Modellen zeigt, dass der Dienst läuft.

7.3 Modellwahl und Werkzeugfähigkeit

Nicht jedes Modell kann Werkzeugaufrufe zuverlässig ausführen. Ein Modell muss das Feld `tools` verarbeiten und strukturierte Aufrufe zurückgeben. Die Modellinformationen geben darüber Auskunft. Die Fähigkeiten eines Modells gehören deshalb zu den Angaben, die vor dem Einsatz geprüft werden.

Eine praktische Prüfung besteht aus einer Aufgabe mit einem einfachen Werkzeug. Gelingt der Aufruf mit korrekten Argumenten, ist das Modell für Werkzeuge geeignet. Scheitert er mehrfach, ist ein anderes Modell oder eine kleinere Werkzeugauswahl zu wählen.

7.4 Die Chat-Schnittstelle und das Feld `tools`

Die Schnittstelle `/api/chat` nimmt ein Modell, eine Liste von Nachrichten und optional eine Liste von Werkzeugen entgegen. Jede Nachricht hat eine Rolle und einen Inhalt. Die Rolle `user` steht für den Benutzer, `assistant` für das Modell und `tool` für das Ergebnis eines Werkzeugaufrufs.

Ein Werkzeug hat in diesem Format den Typ `function`, einen Namen, eine Beschreibung und ein Schema für die Parameter. Das Modell antwortet dann entweder mit Text oder mit einem Feld `tool_calls`, das den Namen und die Argumente des gewünschten Aufrufs nennt.

Der Client muss die Aufrufe ausführen und das Ergebnis als Nachricht mit der Rolle `tool` zurückschicken. Erst danach kann das Modell die Antwort formulieren. Der Ablauf wiederholt sich, solange das Modell Werkzeuge verlangt.

Ollama bietet außerdem einen Endpunkt, der dem Format der OpenAI-Schnittstelle folgt. Er ist für Clients nützlich, die dieses Format bereits sprechen. Dieses Buch verwendet den nativen Endpunkt, weil er direkter ist.

7.5 Umwandlung von MCP-Werkzeugen in das Ollama-Format

Ein MCP-Werkzeug liefert einen Namen, eine Beschreibung und ein Eingabeschema. Das Ollama-Format erwartet dieselben Angaben in einer anderen Hülle. Die Umwandlung ist daher einfach: Der Typ `function` umschließt Name, Beschreibung und Schema.

Das folgende Modul übernimmt die Umwandlung. Es nimmt ein Werkzeug aus dem MCP-Client entgegen und gibt den Eintrag für Ollama zurück:

Datei: projekte/agent_ollama/werkzeuge.py

```
"""Übersetzt MCP-Werkzeuge in das Werkzeugformat von Ollama."""


def mcp_zu_ollama(werkzeug) -> dict:
    """Erwartet ein Tool-Objekt aus dem MCP-Client (name, description,
    inputSchema)."""
    schema = werkzeug.input_schema or {"type": "object", "properties": {}}
    return {
        "type": "function",
        "function": {
            "name": werkzeug.name,
            "description": werkzeug.description or "",
            "parameters": schema,
        },
    }


def alle_umwandeln(werkzeuge) -> list[dict]:
    return [mcp_zu_ollama(w) for w in werkzeuge]
```

Die Umwandlung ist bewusst klein gehalten. Sie verändert weder Namen noch Schema. Ein Werkzeug, das im MCP-Server korrekt definiert ist, bleibt damit auch in Ollama korrekt.

7.6 Kontextfenster, Tokenverbrauch und Ressourcen

Jede Nachricht und jede Werkzeugbeschreibung belegt Plätze im Kontextfenster. Viele Werkzeuge mit langen Beschreibungen füllen es schnell. Ein Agent sollte deshalb nur die Werkzeuge anbieten, die er für eine Aufgabe braucht. Die Zahl der Werkzeuge ist eine Entwurfsentscheidung, keine Nebensache.

Auch die Ergebnisse der Werkzeuge belegen Platz. Ein Werkzeug, das eine große Datei zurückgibt, verdrängt die bisherige Unterhaltung. Die Begrenzung der Ausgabe gehört deshalb zur Gestaltung jedes Werkzeugs. Projekt B begrenzt die gelesene Textmenge aus diesem Grund.

Der Speicherbedarf des Modells hängt von der Größe und der Quantisierung ab, wie Band 4 beschreibt. Der Prozessor oder die Grafikkarte bestimmen die Geschwindigkeit. Die Messung im eigenen System ist die einzige verlässliche Angabe.

7.7 Offline-Betrieb

Ein lokales Modell mit lokalen MCP-Servern arbeitet ohne Netzzugang. Das ist ein Vorteil für Datenschutz und Verfügbarkeit. Die Grenze liegt im Wissen: Ein Modell ohne Netzzugang kennt nur seinen Trainingsstand. Aktuelle Informationen müssen über Werkzeuge in den Kontext gelangen, und das ist nur möglich, wenn ein Werkzeug sie liefert.

7.8 Integration: Ollama mit einem eigenen MCP-Server

Das Projekt des Kapitels besteht aus einem Agenten, der die Schleife ausführt, und einem Client, der die MCP-Werkzeuge bereitstellt. Die Schleife ist im Kapitel 8 ausführlich beschrieben. Hier zeigt der Abschnitt die Verbindung.

Der Agent liest die Werkzeuge vom Server, wandelt sie um und sendet sie mit der Nachricht an Ollama. Hat das Modell einen Aufruf verlangt, führt der Agent ihn über den MCP-Client aus. Das Ergebnis geht zurück an das Modell. Die Funktion `chat` im Modul `ollama_client.py` kapselt die HTTP-Anfrage.

Für die Tests in diesem Buch läuft der Ollama-Dienst als Emulator. Der Emulator liefert festgelegte Antworten mit Werkzeugaufrufen. Er prüft nicht die Qualität eines Modells, sondern ob der Agent die Werkzeuge übergibt, die Aufrufe ausführt und die Ergebnisse zurückschickt. Ob ein echtes Modell die Aufgabe richtig löst, ist mit eigenen Beispielen zu prüfen. Das Buch benennt diese Grenze.

Quellen zu Kapitel 7:

Ollama, Dokumentation der Chat-Schnittstelle und der Werkzeugaufrufe, ollama.com. Vor Verwendung auf den aktuellen Stand prüfen.

Ollama, Dokumentation des OpenAI-kompatiblen Endpunkts. Vor Verwendung prüfen.

Band 4 der Reihe KI PRAXIS, Lokale KI und eigene KI-Systeme.

Kapitel 8: Eigene KI-Agenten mit Python entwickeln

8.1 Aufbau eines Python-Agenten

Ein Python-Agent besteht aus drei Teilen: der Anbindung des Sprachmodells, dem MCP-Client für die Werkzeuge und der Schleife, die beide verbindet. Der Agent ist eine Funktion mit einer Aufgabe als Eingabe und einer Antwort als Ergebnis. Diese Schlichtheit ist beabsichtigt, weil sich damit alle Abläufe nachvollziehen und testen lassen.

8.2 Anbindung des Sprachmodells

Das Modul `ollama_client.py` kapselt die Anfrage an Ollama. Es nimmt das Modell, die Nachrichten und die Werkzeuge entgegen und gibt die Antwortnachricht zurück. Fehler der Verbindung werden in eine eigene Ausnahme verpackt, damit der Agent sie gezielt behandeln kann.

Datei: projekte/agent_ollama/ollama_client.py

```
"""Minimaler Client für die Chat-Schnittstelle von Ollama mit Werkzeugen.

Ollama ist kein MCP-Client. Dieses Modul spricht nur die HTTP-Schnittstelle
von Ollama an. Die Werkzeuge kommen aus dem MCP-Client (siehe agent.py).
"""
import json
import os
import urllib.error
import urllib.request

BASIS = os.environ.get("OLLAMA_URL", "http://127.0.0.1:11434")


class OllamaFehler(Exception):
    pass


def chat(modell: str, nachrichten: list[dict], werkzeuge: list[dict] |
    None = None,
         timeout: float = 120.0) -> dict:
```

```
    """Sendet einen Chat an Ollama und gibt die Antwortnachricht zurück."""
    daten = {"model": modell, "messages": nachrichten, "stream": False}
    if werkzeuge:
        daten["tools"] = werkzeuge
    anfrage = urllib.request.Request(
        BASIS + "/api/chat",
        data=json.dumps(daten).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    try:
        with urllib.request.urlopen(anfrage, timeout=timeout) as antwort:
            return json.loads(antwort.read())["message"]
    except urllib.error.HTTPError as fehler:
        raise OllamaFehler(f"HTTP {fehler.code}:
    {fehler.read().decode(errors='replace')}")
    except urllib.error.URLError as fehler:
        raise OllamaFehler(f"Ollama nicht erreichbar: {fehler.reason}")
```

Die Adresse des Dienstes kommt aus der Umgebungsvariable `OLLAMA_URL`. Ohne Angabe wird der Standardwert des lokalen Dienstes verwendet. Auf diese Weise kann der Test den Emulator einsetzen, ohne den Code zu ändern.

8.3 MCP-Client einbinden

Der Agent startet den MCP-Server als Prozess über stdio und verbindet sich mit dem Client des SDK. Wichtig ist die Umgebung des Serverprozesses. Das SDK gibt ihm nur eine Auswahl sicherer Variablen weiter. Was der Server braucht, etwa das Verzeichnis einer Datei, muss der Agent ausdrücklich übergeben. Die erste Fassung des Agenten vergaß das und schlug in den Tests fehl. Dieser Fehler gehört zu den häufigsten in der Praxis und ist im Kapitel 16 beschrieben.

8.4 Werkzeugauswahl und Aufgabenplanung

Der Agent bietet dem Modell alle Werkzeuge an, die der Server meldet. In größeren Systemen wählt man eine Teilmenge aus, bevor man sie anbietet. Die Auswahl richtet sich nach der Aufgabe. Ein Agent für Systemdiagnose braucht keine Schreibwerkzeuge. Diese Trennung ist eine Sicherheitsmaßnahme und ein Mittel gegen unnötig lange Kontexte.

Die Planung liegt beim Modell. Der Agent gibt ihm die Aufgabe und die Werkzeuge. Welche Schritte es wählt, bestimmt das Modell im Rahmen der Schleife.

8.5 Der Agent Loop

Die Schleife sendet die Nachrichten an das Modell und wartet auf die Antwort. Enthält sie keine Werkzeugaufrufe, endet die Schleife mit dem Text. Enthält sie Aufrufe, führt der Agent jeden davon aus und hängt das Ergebnis als Nachricht mit der Rolle `tool` an. Danach beginnt der nächste Schritt.

Datei: projekte/agent_ollama/agent.py

```
"""Agent mit Ollama-Chat, MCP-Werkzeugen, Schrittgrenze und Freigabe.

Ablauf: Der Agent holt die Werkzeuge vom MCP-Server, sendet sie mit der
Nachricht an Ollama und führt die vom Modell verlangten Aufrufe über den
MCP-Client aus. Das Ergebnis geht als Nachricht mit der Rolle "tool"
zurück an das Modell. Die Schleife endet mit einer Textantwort oder bei
der Schrittgrenze.
"""
import json
import sys
from pathlib import Path

from mcp import Client
from mcp.client.stdio import StdioServerParameters

from ollama_client import chat
from werkzeuge import alle_umwandeln

MAX_SCHRITTE = 5
# Werkzeuge, die vor der Ausführung eine Freigabe brauchen.
FREIGABE_PFLICHT: set[str] = {"schreibe_datei"}
```

```
class SchrittgrenzeErreicht(RuntimeError):
    pass


async def ausfuehren(modell: str, server_skript: Path, aufgabe: str,
                     freigabe=lambda name, args: False,
                     env: dict[str, str] | None = None) -> str:
    # Das SDK gibt dem Serverprozess nur eine Auswahl sicherer Variablen
    weiter.
    # Alles, was der Server braucht, muss hier ausdrücklich übergeben
    werden.
    params = StdioServerParameters(command=sys.executable,
                                   args=[str(server_skript)], env=env)
    antwort_text: str | None = None
    async with Client(params) as mcp_client:
        werkzeuge = (await mcp_client.list_tools()).tools
        ollama_werkzeuge = alle_umwandeln(werkzeuge)
        nachrichten = [{"role": "user", "content": aufgabe}]

        for schritt in range(1, MAX_SCHRITTE + 1):
            antwort = chat(modell, nachrichten, ollama_werkzeuge)
            nachrichten.append(antwort)
            aufrufe = antwort.get("tool_calls") or []
            if not aufrufe:
                antwort_text = antwort.get("content", "")
                break
            for aufruf in aufrufe:
                name = aufruf["function"]["name"]
                args = aufruf["function"].get("arguments") or {}
                if isinstance(args, str):
                    args = json.loads(args)
                if name in FREIGABE_PFLICHT and not freigabe(name, args):
                    text = f"Aktion {name} ohne Freigabe nicht ausgeführt."
                else:
                    ergebnis = await mcp_client.call_tool(name, args)
                    text = ergebnis.content[0].text if ergebnis.content
    else ""
                    if ergebnis.is_error:
                        text = "Fehler: " + text
                nachrichten.append({"role": "tool", "content": text,
    "name": name})
    # Die Ausnahme wird außerhalb des Client-Blocks ausgelöst, damit sie
    # nicht in eine ExceptionGroup verpackt wird.
    if antwort_text is None:
        raise SchrittgrenzeErreicht("Schrittgrenze erreicht, ohne dass
    der Agent antwortete")
    return antwort_text
```

Die Schleife hat drei Schutzmechanismen. Die Schrittgrenze verhindert endlose Wiederholungen. Die Freigabe verlangt die Zustimmung für Aktionen aus der Liste `FREIGABE_PFLICHT`. Fehlerergebnisse werden als Text mit dem Präfix „Fehler:“ an das Modell

zurückgegeben, damit es darauf reagieren kann.

8.6 Mehrstufige Aufgaben und Konversationshistorie

Eine mehrstufige Aufgabe entsteht, wenn das Modell die Ergebnisse eines Schritts für den nächsten braucht. Die Historie enthält alle Nachrichten der Sitzung. Sie wächst mit jedem Schritt. Kapitel 11 zeigt, wie man sie begrenzt und dauerhaft speichert.

8.7 Wiederholungen und Fehlerbehandlung

Ein Werkzeug kann vorübergehend fehlschlagen, etwa weil eine Netzverbindung kurz gestört ist. Eine Wiederholung mit wachsenden Wartezeiten hilft in solchen Fällen. Sie ist nur bei Werkzeugen sinnvoll, die beim wiederholten Aufruf keinen Schaden anrichten. Ein schreibendes Werkzeug darf nicht blind wiederholt werden, wie Band 3 am Beispiel der Zahlung zeigt.

8.8 Tests des Agenten

Die Tests in `tests/test_agent.py` prüfen den Ablauf ohne echtes Modell und ohne Netzzugang. Vier Fälle sind abgedeckt. Der erste prüft, dass ein Werkzeugaufruf ausgeführt und das Ergebnis zurückgegeben wird. Die beiden folgenden prüfen die Freigabe: Ohne Zustimmung wird nicht geschrieben, mit Zustimmung schon. Der vierte prüft die Schrittgrenze.

Ausgabe aus dem Testlauf in der Referenzumgebung: `4 passed`.

Quellen zu Kapitel 8:

MCP Python SDK, Version 2.x, Client und stdio-Transport, github.com/modelcontextprotocol/python-sdk.

Ollama, Dokumentation der Chat-Schnittstelle, ollama.com.

Python-Dokumentation: asyncio, urllib.request. docs.python.org.

Kapitel 9: KI-Agenten für Linux und Windows

9.1 Linux: Systeminformationen, Prozessor, Speicher, Festplatten

Linux stellt Systemangaben als Dateien unter `/proc` und `/sys` bereit. Das ist für einen Agenten vorteilhaft, weil er die Werte ohne zusätzliche Programme lesen kann. Projekt A aus Kapitel 6 nutzt diesen Weg. Die Werkzeuge liefern Arbeitsspeicher, Prozessorauslastung, Festplattenbelegung und Betriebszeit.

Die Messung der Prozessorauslastung braucht zwei Stichproben, weil die Datei `/proc/stat` die Zeit seit dem Start zählt. Die Differenz der beiden Werte ergibt die Auslastung im Intervall. Das Kapitel 6 hat diese Berechnung mit festen Messwerten getestet.

Für Prozesse gilt dasselbe Prinzip. Jeder Prozess hat ein Verzeichnis unter `/proc` mit seinem Namen und seinen Angaben. Ein Werkzeug, das Prozesse auflistet, liest diese Verzeichnisse. Es muss dabei damit rechnen, dass ein Prozess während des Lesens endet. Ein solcher Fehler ist erwartbar und darf den Server nicht zum Absturz bringen.

9.2 Linux: Netzwerkstatus, Dienste, Systemprotokolle

Netzwerkangaben erhält man über `/proc/net` oder über die Standardbibliothek. Projekt C aus Kapitel 6 prüft Namen, Ports und Erreichbarkeit, ohne Einstellungen zu verändern. Für die Abfrage der Dienste nutzt man unter Debian den Dienstverwalter systemd. Der Befehl `systemctl` liefert den Status eines Dienstes. Der Agent sollte ihn nur mit einer festen Argumentliste und nur lesend aufrufen.

Systemprotokolle liegen unter Debian im Journal, das mit `journalctl` gelesen wird. Auch hier gilt: Ein Werkzeug darf nur lesen, und die

Ausgabe muss begrenzt werden, weil Protokolle schnell sehr groß werden. Protokolle enthalten außerdem oft personenbezogene Angaben. Band 8 hat dazu die Schwärzung beschrieben.

9.3 Linux: Dateien und kontrollierte Befehle

Dateiwerkzeuge gehören zu den nützlichsten und zugleich gefährlichsten Werkzeugen eines Agenten. Projekt B aus Kapitel 6 zeigt die Grundregeln: Pfade werden aufgelöst und innerhalb eines Arbeitsordners geprüft, Schreiben verlangt eine ausdrückliche Bestätigung.

Befehle aus der Shell bieten Möglichkeiten, die ein Werkzeug nicht vorhersehen kann. Ein Werkzeug, das einen beliebigen Text als Befehl ausführt, gibt dem Modell die volle Kontrolle über das System. Das Buch verwendet deshalb keine Werkzeuge dieser Art. Wo ein Systembefehl nötig ist, ist er fest verdrahtet, mit einer festen Argumentliste gestartet und nur lesend.

9.4 Windows: PowerShell, Dienste, Ereignisprotokolle

Dieser Abschnitt ist in der Referenzumgebung nicht geprüft. Die Angaben beschreiben den üblichen Weg und müssen auf einem Windows-11-System getestet werden, bevor sie eingesetzt werden.

Unter Windows liefern PowerShell-Befehle die Systemangaben. Der Befehl `Get-Service` listet Dienste mit ihrem Status, `Get-EventLog` oder `Get-WinEvent` liest Ereignisprotokolle. Ein MCP-Server für Windows kann diese Befehle über das Programm `powershell.exe` mit einer festen Argumentliste aufrufen. Die Ausgabe sollte als strukturierte Daten zurückkommen, etwa im JSON-Format, damit das Modell sie zuverlässig auswertet.

Die Rechte sind unter Windows anders verteilt als unter Linux. Ein Dienst lässt sich nur mit Administratorrechten ändern. Ein Agent sollte

deshalb ausschließlich lesende Dienstabfragen erhalten, bis eine bewusste Entscheidung für weitere Rechte getroffen wurde. Die Anforderungen an Ereignisprotokolle können von Richtlinien des Unternehmens eingeschränkt sein.

9.5 Sichere Bereitstellung von Betriebssystemfunktionen

Die Regel für alle Betriebssystemwerkzeuge lautet: Jedes Werkzeug tut genau eine Sache, liest oder schreibt nur, was es braucht, und begrenzt seine Ausgabe. Ein Werkzeug, das Prozesse beendet, ist eine andere Klasse als eines, das sie auflistet, und braucht eine eigene Freigabe.

Die Trennung in Rechtestufen aus Kapitel 13 setzt diese Regel um. Lesewerkzeuge sind frei, Schreibwerkzeuge verlangen Bestätigung, Administrationswerkzeuge brauchen eine zusätzliche Prüfung.

Quellen zu Kapitel 9:

Linux-Kernel-Dokumentation zu procfs, kernel.org.

systemd-Dokumentation zu systemctl und journalctl, freedesktop.org.

Microsoft-Dokumentation zu PowerShell, Get-Service und Get-WinEvent, learn.microsoft.com. Vor Verwendung auf den aktuellen Stand prüfen.

Kapitel 10: Datenbanken, APIs und externe Dienste

10.1 HTTP, JSON und REST

Eine Programmierschnittstelle im Web nimmt Anfragen über HTTP entgegen und antwortet mit Daten. Häufig ist das Format JSON. Eine REST-Schnittstelle organisiert ihre Adressen nach Ressourcen, und die HTTP-Methode legt die Handlung fest: GET liest, POST legt an, PUT und PATCH ändern, DELETE entfernt.

Ein Agent ruft solche Schnittstellen über ein Werkzeug auf. Das Werkzeug kennt die Adresse, die nötigen Parameter und das Format der Antwort. Das Modell sieht nur die Beschreibung und die Ergebnisse.

10.2 Authentifizierung und API-Schlüssel

Viele Schnittstellen verlangen einen Schlüssel, der jeder Anfrage beigefügt wird. Ein Schlüssel ist wie ein Passwort zu behandeln. Er gehört in eine Umgebungsvariable oder einen Geheimnisspeicher, nie in den Quellcode oder in eine Datei, die in die Versionsverwaltung gelangt. Band 8 hat das Thema ausführlich behandelt.

Ein Werkzeug, das im Namen eines Benutzers auf einen Dienst zugreift, sollte nur die Rechte nutzen, die der Benutzer dem Werkzeug gegeben hat. Das ist das Prinzip der minimalen Rechte aus Kapitel 13.

10.3 SQLite als Gedächtnis- und Datenspeicher

SQLite ist eine Datenbank, die als einzelne Datei vorliegt und keinen Serverdienst braucht. Sie gehört zur Standardbibliothek von Python. Für Agenten, die Notizen, Aufgaben oder Verläufe speichern, ist sie eine einfache und zuverlässige Wahl.

Eine Verbindung im Nur-Lese-Modus verhindert Schreibvorgänge auf Datenbankebene. Die Angabe `mode=ro` in der Verbindungs-Adresse erzwingt das. Diese Schutzschicht wirkt auch dann, wenn eine Prüfung im Programm umgangen wird. Das Projekt D des Kapitels nutzt diese Eigenschaft.

10.4 PostgreSQL

PostgreSQL ist ein Datenbankserver, den viele Organisationen einsetzen. Er bietet Benutzerverwaltung, Berechtigungen auf Tabellenebene und Mehrbenutzerbetrieb. Ein Agent, der auf PostgreSQL zugreift, sollte eine eigene Datenbankrolle mit wenigen Rechten erhalten. Die Einrichtung eines Servers ist nicht Teil dieses Buches. Die Verbindung in diesem Kapitel nutzt SQLite, das Prinzip gilt aber für beide.

Dieser Abschnitt ist in der Referenzumgebung nicht als Projekt geprüft. Er beschreibt die Rolle und die Rechte, die ein Agent dafür braucht.

10.5 MCP-Server für Datenbanken

Ein MCP-Server für eine Datenbank stellt Werkzeuge bereit: die Tabellen auflisten, eine Abfrage ausführen, einen Eintrag anlegen. Die Auswahl dieser Werkzeuge bestimmt die Sicherheit. Abfragen mit freiem SQL sind mächtig. Ein Werkzeug, das nur SELECT zulässt, verhindert Änderungen, deckt aber nicht jede Abfrage ab, die man sich wünscht.

Das Projekt D kombiniert beides. Die Abfrage läuft über eine Verbindung im Nur-Lese-Modus und erlaubt nur eine einzelne SELECT-Anweisung. Das Schreiben ist auf eine Liste freigegebener Tabellen beschränkt und verlangt eine Bestätigung.

10.6 Validierung, Timeouts und Rate Limiting

Jede Eingabe wird geprüft, bevor sie das System erreicht. Tabellennamen und Spaltennamen können nicht als Parameter übergeben werden. Sie werden deshalb gegen eine feste Liste geprüft, bevor sie in eine Anweisung gelangen. Werte dagegen gehen als gebundene Parameter an die Datenbank, damit sie nicht als Befehl gelesen werden.

Ein Timeout begrenzt die Wartezeit auf eine Antwort. Ohne Timeout kann ein Agent an einer hängenden Schnittstelle unbegrenzt warten. Die Begrenzung der Anfragen pro Zeitraum verhindert, dass ein Modell eine Schnittstelle überlastet, etwa durch eine Schleife.

10.7 Projekt D: Datenbank- und API-Server

Projektziel: Ein MCP-Server, der eine SQLite-Datenbank lesend abfragt, freigegebene Tabellen beschreibt und ausgewählte JSON-Schnittstellen abruft.

Voraussetzungen: Python 3.10 oder neuer. Keine externe Datenbank, keine Netzverbindung nach außen. Die Tests verwenden einen lokalen Testserver.

Verzeichnisstruktur:

```
projekte/d_datenbank/
  datenbank_server.py
  test_datenbank.py
```

Quellcode des Servers:

Datei: projekte/d_datenbank/datenbank_server.py

```
"""Datenbank- und API-Server (Projekt D, Kapitel 10).

- Abfragen laufen über eine Verbindung im Nur-Lese-Modus (SQLite mode=ro).
- Schreiben verlangt eine ausdrückliche Bestätigung und nur erlaubte
    Tabellen.
- HTTP-Abfragen gehen nur an Hosts aus der Liste MCP_API_HOSTS.
"""
```

```
import json
import os
import re
import sqlite3
import time
import urllib.parse
import urllib.request
from pathlib import Path

from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError

mcp = MCPServer("datenbank")

NAME_MUSTER = re.compile(r"^[A-Za-z_][A-Za-z0-9_]{0,63}$")
ERLAUBTE_TABELLEN = {"notizen", "aufgaben"}
MAX_ANTWORT = 200_000
API_TIMEOUT = 5.0
_zeiten: list[float] = []
MAX_API_AUFRUFE_PRO_MINUTE = 5


def db_pfad() -> Path:
    return Path(os.environ.get("MCP_DB", "daten.db")).resolve()


def nur_lesen() -> sqlite3.Connection:
    """Öffnet die Datenbank schreibgeschützt."""
    uri = f"file:{db_pfad()}?mode=ro"
    return sqlite3.connect(uri, uri=True)


def schreiben() -> sqlite3.Connection:
    return sqlite3.connect(db_pfad())


def name_pruefen(name: str) -> str:
    if not NAME_MUSTER.fullmatch(name):
        raise ToolError(f"Ungültiger Name: {name!r}")
    return name


def tabelle_pruefen(name: str) -> str:
    name_pruefen(name)
    if name not in ERLAUBTE_TABELLEN:
        raise ToolError(f"Tabelle nicht freigegeben: {name}")
    return name


@mcp.tool()
def tabellen() -> list[str]:
    """Listet die Tabellen der Datenbank auf."""
    with nur_lesen() as db:
        zeilen = db.execute(
            "SELECT name FROM sqlite_master WHERE type='table' ORDER BY
```

```
    name")
        return [z[0] for z in zeilen]


@mcp.tool()
def abfrage(sql: str, grenze: int = 50) -> list[dict]:
    """Führt eine einzelne SELECT-Abfrage aus und gibt höchstens grenze
    Zeilen zurück."""
    text = sql.strip().rstrip(";")
    if ";" in text or not text.upper().startswith("SELECT"):
        raise ToolError("Nur eine einzelne SELECT-Abfrage ist erlaubt")
    grenze = min(max(grenze, 1), 500)
    with nur_lesen() as db:
        cursor = db.execute(text)
        spalten = [s[0] for s in cursor.description]
        return [dict(zip(spalten, zeile)) for zeile in
    cursor.fetchmany(grenze)]


@mcp.tool()
def eintrag_anlegen(tabelle: str, text: str, bestaetigt: bool = False) ->
    str:
    """Legt einen Eintrag mit einem Textfeld an. Erfordert Bestätigung."""
    if not bestaetigt:
        raise ToolError("Schreiben ohne Bestätigung abgelehnt
    (bestaetigt=True nötig)")
    tabelle_pruefen(tabelle)
    with schreiben() as db:
        db.execute(f"INSERT INTO {tabelle} (inhalt) VALUES (?)", (text,))
    return f"Eintrag in {tabelle} angelegt"


def _api_grenze() -> None:
    jetzt = time.time()
    _zeiten[:] = [t for t in _zeiten if jetzt - t < 60]
    if len(_zeiten) >= MAX_API_AUFRUFE_PRO_MINUTE:
        raise ToolError("Zu viele API-Aufrufe in der letzten Minute")
    _zeiten.append(jetzt)


@mcp.tool()
def api_lesen(url: str) -> dict:
    """Ruft eine JSON-Schnittstelle ab. Nur Hosts aus MCP_API_HOSTS sind
    erlaubt."""
    erlaubt = {h.strip() for h in os.environ.get("MCP_API_HOSTS",
    "").split(",") if h.strip()}
    host = urllib.parse.urlparse(url)
    if host.scheme not in ("http", "https") or host.hostname not in erlaubt:
        raise ToolError(f"Host nicht freigegeben: {host.hostname}")
    _api_grenze()
    anfrage = urllib.request.Request(url, headers={"Accept":
    "application/json"})
    try:
        with urllib.request.urlopen(anfrage, timeout=API_TIMEOUT) as
    antwort:
```

```
            roh = antwort.read(MAX_ANTWORT + 1)
    except OSError as fehler:
        raise ToolError(f"Schnittstelle nicht erreichbar: {fehler}")
    if len(roh) > MAX_ANTWORT:
        raise ToolError("Antwort zu groß")
    try:
        return {"status": "ok", "daten": json.loads(roh)}
    except json.JSONDecodeError:
        raise ToolError("Antwort ist kein gültiges JSON")


if __name__ == "__main__":
    mcp.run()
```

Quellcode der Tests:

Datei: projekte/d_datenbank/test_datenbank.py

```
import sqlite3
import threading
from http.server import BaseHTTPRequestHandler, HTTPServer

import pytest

import datenbank_server as d
from mcp.server.mcpserver.exceptions import ToolError


@pytest.fixture
def db(tmp_path, monkeypatch):
    pfad = tmp_path / "test.db"
    con = sqlite3.connect(pfad)
    con.execute("CREATE TABLE notizen (id INTEGER PRIMARY KEY, inhalt
    TEXT)")
    con.execute("CREATE TABLE geheim (id INTEGER PRIMARY KEY, inhalt TEXT)")
    con.execute("INSERT INTO notizen (inhalt) VALUES ('erste')")
    con.commit()
    con.close()
    monkeypatch.setenv("MCP_DB", str(pfad))
    d._zeiten.clear()
    return pfad


def test_tabellen_liste(db):
    assert d.tabellen() == ["geheim", "notizen"]


def test_select_liefert_zeilen(db):
    assert d.abfrage("SELECT inhalt FROM notizen") == [{"inhalt": "erste"}]


@pytest.mark.parametrize("sql", [
    "DELETE FROM notizen",
    "SELECT 1; DROP TABLE notizen",
```

```
    "PRAGMA table_info(notizen)",
])
def test_nicht_select_wird_abgelehnt(db, sql):
    with pytest.raises(ToolError):
        d.abfrage(sql)


def test_schreibverbindung_ist_schreibgeschuetzt(db):
    # Auch wenn eine Abfrage die Prüfung umgeht, verhindert die
    Verbindung das Schreiben.
    with pytest.raises(sqlite3.OperationalError):
        with d.nur_lesen() as con:
            con.execute("INSERT INTO notizen (inhalt) VALUES ('x')")


def test_eintrag_ohne_bestaetigung_abgelehnt(db):
    with pytest.raises(ToolError):
        d.eintrag_anlegen("notizen", "neu")


def test_eintrag_mit_bestaetigung(db):
    d.eintrag_anlegen("notizen", "neu", bestaetigt=True)
    assert len(d.abfrage("SELECT id FROM notizen")) == 2


def test_tabelle_nicht_freigegeben(db):
    with pytest.raises(ToolError):
        d.eintrag_anlegen("geheim", "x", bestaetigt=True)


@pytest.mark.parametrize("name", ["notizen; DROP", "a b", "1abc"])
def test_ungueltige_namen(name):
    with pytest.raises(ToolError):
        d.name_pruefen(name)


@pytest.fixture
def api_server():
    class H(BaseHTTPRequestHandler):
        def do_GET(self):
            body = b'{"wert": 42}' if self.path == "/ok" else b"kein json"
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def log_message(self, *a):
            pass

    server = HTTPServer(("127.0.0.1", 0), H)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    yield f"http://127.0.0.1:{server.server_address[1]}"
    server.shutdown()
```

```
def test_api_erlaubter_host(api_server, monkeypatch):
    monkeypatch.setenv("MCP_API_HOSTS", "127.0.0.1")
    d._zeiten.clear()
    assert d.api_lesen(api_server + "/ok") == {"status": "ok", "daten":
    {"wert": 42}}


def test_api_fremder_host_abgelehnt(monkeypatch):
    monkeypatch.setenv("MCP_API_HOSTS", "example.org")
    with pytest.raises(ToolError):
        d.api_lesen("http://127.0.0.1:9/ok")


def test_api_kein_json(api_server, monkeypatch):
    monkeypatch.setenv("MCP_API_HOSTS", "127.0.0.1")
    d._zeiten.clear()
    with pytest.raises(ToolError):
        d.api_lesen(api_server + "/kaputt")


def test_api_grenze_pro_minute(api_server, monkeypatch):
    monkeypatch.setenv("MCP_API_HOSTS", "127.0.0.1")
    d._zeiten.clear()
    for _ in range(d.MAX_API_AUFRUFE_PRO_MINUTE):
        d.api_lesen(api_server + "/ok")
    with pytest.raises(ToolError):
        d.api_lesen(api_server + "/ok")
```

Starten und prüfen:

```
cd projekte/d_datenbank
source ../../.venv/bin/activate
python -m pytest -q test_datenbank.py
```

Erwartetes Ergebnis: `16 passed`.

Erklärung: Die Funktion `nur_lesen` öffnet die Datenbank mit der Adresse `file:...?mode=ro`. Eine Schreibanweisung schlägt dort fehl, auch wenn die Textprüfung in `abfrage` sie durchlässt. Der Test prüft diese zweite Schutzschicht ausdrücklich. Die Funktion `api_lesen` akzeptiert nur Hosts aus der Umgebungsvariable `MCP_API_HOSTS`. Die Grenze von fünf Aufrufen pro Minute begrenzt die Last.

Protokolltest: Der Test startet den Server als Prozess und prüft Abfrage, Schreibschutz und Bestätigung. Ausgabe: `datenbank: Abfrage, Schreibschutz und Bestätigung über das Protokoll geprüft`.

Troubleshooting: Meldet der Server „attempt to write a readonly database“, wurde eine Schreibanweisung über die Nur-Lese-Verbindung ausgeführt. Das ist gewollt. Schreibvorgänge laufen über `eintrag_anlegen`.

Kapitel 11: Gedächtnis und Wissensmanagement

11.1 Kurzzeitgedächtnis und Langzeitgedächtnis

Ein Agent hat zwei Arten von Gedächtnis. Das Kurzzeitgedächtnis ist der Verlauf der laufenden Sitzung. Es passt in das Kontextfenster des Modells und verschwindet, wenn die Sitzung endet. Das Langzeitgedächtnis besteht aus Informationen, die über Sitzungen hinweg erhalten bleiben. Es liegt außerhalb des Modells, etwa in einer Datei oder Datenbank.

Das Modell erinnert sich nicht selbst. Erst wenn das Programm die gespeicherten Notizen in den Kontext legt, kann das Modell sie verwenden. Das gilt für jede Form des Gedächtnisses und ist der wichtigste Gedanke dieses Kapitels.

11.2 Persistente Speicherung

Persistenz heißt, dass Daten einen Programmabbruch und einen Neustart überstehen. Eine Datei oder eine Datenbank erfüllt das. Die Wahl hängt von der Menge und der Abfragehäufigkeit ab. Für wenige hundert Notizen genügt eine SQLite-Datei. Für sehr große Mengen oder viele gleichzeitige Nutzer braucht man einen Datenbankserver.

11.3 Embeddings und Vektorsuche

Eine Suche nach Schlüsselwörtern findet nur Notizen, die dieselben Wörter enthalten. Eine Suche nach Bedeutung erfordert Embeddings. Ein Embedding ist ein Vektor aus Zahlen, der die Bedeutung eines Textes beschreibt. Texte mit ähnlicher Bedeutung liegen in diesem Vektorraum nahe beieinander. Band 2 hat das Verfahren eingeführt, Band 4 seine lokale Umsetzung beschrieben.

Das Projekt E dieses Kapitels verwendet bewusst keine Embeddings. Ein Embedding-Modell muss geladen und seine Qualität geprüft werden. Das ist in der Referenzumgebung nicht möglich. Die Suche arbeitet mit Schlüsselwörtern und zählt die Treffer. Der Abschnitt 11.6 zeigt, wie man eine Vektorsuche ergänzt.

11.4 Retrieval-Augmented Generation im Agenten

Retrieval-Augmented Generation bezeichnet das Verfahren, relevante Textabschnitte abzurufen und dem Modell als Kontext mitzugeben. Im Agenten ist das ein Werkzeug: Das Modell ruft die Suche auf, erhält die Treffer und formuliert die Antwort. Die Herkunft jeder Aussage bleibt nachvollziehbar, wenn die Treffer mit ihrem Schlüssel erscheinen.

11.5 Grenzen und Risiken des Agentengedächtnisses

Ein Gedächtnis speichert alles, was man ihm gibt, auch Fehler und Angriffe. Eine manipulierte Notiz kann das Verhalten des Agenten in späteren Sitzungen verändern. Dieses Risiko heißt Gedächtnisvergiftung. Die Gegenmaßnahme besteht darin, nur bestätigte Inhalte aufzunehmen und jede Notiz mit Herkunft und Zeitpunkt zu versehen.

Ein Gedächtnis kann außerdem personenbezogene Angaben enthalten, die niemand mehr sehen oder löschen möchte. Das Löschwerkzeug aus Projekt E verlangt deshalb eine Bestätigung und entfernt die Notiz vollständig.

11.6 Projekt E: Persistentes Gedächtnissystem

Projektziel: Ein MCP-Server, der Notizen dauerhaft speichert, nach Schlüsselwörtern sucht und auf Wunsch löscht.

Voraussetzungen: Python 3.10 oder neuer. Keine externe Datenbank.

Verzeichnisstruktur:

```
projekte/e_gedaechtnis/
  gedaechtnis_server.py
  test_gedaechtnis.py
```

Quellcode des Servers:

Datei: projekte/e_gedaechtnis/gedaechtnis_server.py

```
"""Persistentes Gedächtnis (Projekt E, Kapitel 11).

Speichert Notizen in einer SQLite-Datei. Die Suche arbeitet mit
    Schlüsselwörtern
und Trefferzählung. Embeddings sind nicht enthalten, siehe Kapitel 11.
"""
import os
import re
import sqlite3
from pathlib import Path

from mcp.server.mcpserver import MCPServer
from mcp.server.mcpserver.exceptions import ToolError

mcp = MCPServer("gedaechtnis")
MAX_TEXT = 4000


def pfad() -> Path:
    return Path(os.environ.get("MCP_GEDAECHTNIS",
    "gedaechtnis.db")).resolve()


def verbinden() -> sqlite3.Connection:
    db = sqlite3.connect(pfad())
    db.execute("""CREATE TABLE IF NOT EXISTS notizen (
        schluessel TEXT PRIMARY KEY,
        text TEXT NOT NULL,
        schlagworte TEXT NOT NULL DEFAULT '',
        angelegt TEXT NOT NULL DEFAULT (datetime('now')))""")
    return db


def woerter(text: str) -> set[str]:
    return {w for w in re.findall(r"\w+", text.lower()) if len(w) > 2}


@mcp.tool()
def merke(schluessel: str, text: str, schlagworte: str = "") -> str:
    """Speichert eine Notiz unter einem Schlüssel. Vorhandene Notizen
    werden ersetzt."""
    if not re.fullmatch(r"[A-Za-z0-9_\-]{1,64}", schluessel):
        raise ToolError("Schlüssel: nur Buchstaben, Ziffern, _ und -
```

```
    (max. 64 Zeichen)")
    if not text.strip() or len(text) > MAX_TEXT:
        raise ToolError(f"Text muss zwischen 1 und {MAX_TEXT} Zeichen
    lang sein")
    with verbinden() as db:
        db.execute("INSERT OR REPLACE INTO notizen (schluessel, text,
    schlagworte) "
                   "VALUES (?, ?, ?)", (schluessel, text,
    schlagworte.lower()))
    return f"gespeichert: {schluessel}"


@mcp.tool()
def finde(suchtext: str, grenze: int = 5) -> list[dict]:
    """Findet Notizen, die die Wörter der Suche enthalten. Sortiert nach
    Trefferzahl."""
    suche = woerter(suchtext)
    if not suche:
        raise ToolError("Die Suche braucht mindestens ein Wort mit drei
    oder mehr Zeichen")
    treffer = []
    with verbinden() as db:
        for schluessel, text, schlagworte in db.execute(
                "SELECT schluessel, text, schlagworte FROM notizen"):
            inhalt = woerter(text) | woerter(schlagworte)
            punkte = len(suche & inhalt)
            if punkte:
                treffer.append({"schluessel": schluessel, "treffer":
    punkte, "text": text})
    treffer.sort(key=lambda t: (-t["treffer"], t["schluessel"]))
    return treffer[:min(max(grenze, 1), 20)]


@mcp.tool()
def vergiss(schluessel: str, bestaetigt: bool = False) -> str:
    """Löscht eine Notiz. Erfordert Bestätigung."""
    if not bestaetigt:
        raise ToolError("Löschen ohne Bestätigung abgelehnt
    (bestaetigt=True nötig)")
    with verbinden() as db:
        cursor = db.execute("DELETE FROM notizen WHERE schluessel = ?",
    (schluessel,))
        if cursor.rowcount == 0:
            raise ToolError(f"Keine Notiz unter {schluessel!r}")
    return f"gelöscht: {schluessel}"


if __name__ == "__main__":
    mcp.run()
```

Quellcode der Tests:

Datei: projekte/e_gedaechtnis/test_gedaechtnis.py

```
import pytest

import gedaechtnis_server as g
from mcp.server.mcpserver.exceptions import ToolError


@pytest.fixture(autouse=True)
def speicher(tmp_path, monkeypatch):
    monkeypatch.setenv("MCP_GEDAECHTNIS", str(tmp_path / "g.db"))


def test_merke_und_finde():
    g.merke("kaffee", "Der Nutzer trinkt Kaffee ohne Zucker", "vorlieben")
    treffer = g.finde("Kaffee Zucker")
    assert treffer[0]["schluessel"] == "kaffee" and treffer[0]["treffer"]
    == 2


def test_schlagworte_werden_mitgesucht():
    g.merke("arbeit", "Projektbericht bis Freitag", "termine")
    assert g.finde("termine")[0]["schluessel"] == "arbeit"


def test_sortierung_nach_trefferzahl():
    g.merke("a", "Python Liste")
    g.merke("b", "Python Liste Schleife Test")
    treffer = g.finde("Python Liste Schleife")
    assert [t["schluessel"] for t in treffer] == ["b", "a"]


def test_ersetzen_einer_notiz():
    g.merke("x", "alter Text")
    g.merke("x", "neuer Text")
    assert g.finde("neuer")[0]["text"] == "neuer Text"
    assert g.finde("alter") == []


def test_persistenz_ueber_neue_verbindung():
    g.merke("dauer", "bleibt erhalten")
    assert g.finde("erhalten")[0]["schluessel"] == "dauer"


@pytest.mark.parametrize("schluessel", ["a b", "../x", ""])
def test_ungueltiger_schluessel(schluessel):
    with pytest.raises(ToolError):
        g.merke(schluessel, "text")


def test_leerer_text_abgelehnt():
    with pytest.raises(ToolError):
        g.merke("k", "   ")


def test_suche_ohne_brauchbare_woerter():
    with pytest.raises(ToolError):
```

```
        g.finde("a an")


def test_vergessen_ohne_bestaetigung():
    g.merke("weg", "bleibt")
    with pytest.raises(ToolError):
        g.vergiss("weg")
    assert g.finde("bleibt")


def test_vergessen_mit_bestaetigung():
    g.merke("weg", "verschwindet")
    g.vergiss("weg", bestaetigt=True)
    assert g.finde("verschwindet") == []


def test_vergessen_unbekannt():
    with pytest.raises(ToolError):
        g.vergiss("gibt-es-nicht", bestaetigt=True)
```

Starten und prüfen:

```
cd projekte/e_gedaechtnis
source ../../.venv/bin/activate
python -m pytest -q test_gedaechtnis.py
```

Erwartetes Ergebnis: `13 passed`.

Erklärung: Die Suche zerlegt Text in Wörter, lässt sehr kurze Wörter weg und zählt die gemeinsamen Wörter mit der Notiz. Die Sortierung stellt die Notiz mit den meisten Treffern nach oben. Die Schlagwörter fließen in die Suche ein. Persistenz entsteht durch die SQLite-Datei, die der Server bei jedem Zugriff öffnet. Die Tests prüfen, dass eine Notiz nach einer neuen Verbindung noch vorhanden ist.

Erweiterung: Für eine Vektorsuche wird die Funktion `woerter` durch ein Embedding-Modell ersetzt. Die Trefferzählung wird dann durch einen Ähnlichkeitswert ersetzt. Die Struktur des Werkzeugs bleibt gleich. Diese Erweiterung braucht ein geprüftes Modell und ist nicht Teil der getesteten Fassung.

Protokolltest: Der Test speichert eine Notiz und findet sie über das Protokoll. Ausgabe: `gedächtnis: Speichern und Finden über das Protokoll geprüft`.

Quellen zu Kapitel 10 und 11:

Python-Dokumentation: sqlite3, docs.python.org. Die Angabe zu `mode=ro` in der URI-Form stammt aus der Dokumentation des Moduls.

SQLite-Dokumentation, sqlite.org/docs.html.

Model Context Protocol, Spezifikation Revision 2026-07-28, Abschnitte zu Tools und Ressourcen.

Kapitel 12: Multi-Agent-Systeme

12.1 Grundlagen von Multi-Agent-Systemen

Ein Multi-Agent-System besteht aus mehreren Agenten, die zusammenarbeiten. Jeder Agent hat eine eigene Aufgabe, eigene Werkzeuge und oft einen eigenen Auftrag. Die Zusammenarbeit entsteht über den Austausch von Ergebnissen. Die Idee ist alt, in der Informatik wurde sie lange vor den Sprachmodellen untersucht. Neu ist, dass ein Sprachmodell die Aufgabe eines einzelnen Agenten übernehmen kann.

Ein einzelner Agent mit vielen Werkzeugen wird schnell unübersichtlich. Er verliert den Überblick über seine Rechte, und sein Kontext füllt sich mit Beschreibungen, die er für die aktuelle Aufgabe nicht braucht. Eine Aufteilung in spezialisierte Agenten begrenzt beides.

12.2 Spezialisierte Agenten

Ein spezialisierter Agent kennt eine Aufgabe gut und hat nur die Werkzeuge, die sie braucht. Der Systemadministrator liest Systemangaben. Der Rechercheagent sucht in Dateien. Der Analyseagent bewertet Ergebnisse und hat keinen Zugriff auf Werkzeuge. Diese Trennung ist eine Sicherheitsmaßnahme: Ein Fehler im Rechercheagenten kann keine Systemangaben verändern, weil er sie nicht aufrufen darf.

Die Auswahl der Rollen folgt aus den Aufgaben, nicht aus der Technik. Man beginnt mit der Frage, welche Teilaufgaben es gibt und welche Rechte jede braucht. Erst danach ordnet man Werkzeuge zu.

12.3 Rollen und Verantwortlichkeiten

Jeder Agent braucht eine Rolle im Sinne des Berechtigungsmodells aus Kapitel 13. Die Rolle bestimmt, welche Stufen von Aktionen er ausführen darf. Eine Rolle ist nicht dasselbe wie eine Aufgabe. Zwei Agenten können dieselbe Rolle haben, aber unterschiedliche Werkzeuge nutzen.

Verantwortlich bleibt der Mensch, der das System betreibt. Ein Agent ist kein Verantwortungsträger. Das Protokoll muss zeigen, welcher Agent welche Aktion ausgelöst hat, damit eine Entscheidung später nachvollzogen werden kann.

12.4 Agentenkommunikation und Aufgabenverteilung

Agenten tauschen Ergebnisse aus. Im Buch geschieht das über einen Koordinator, der die Ergebnisse eines Agenten als Eingabe an den nächsten weiterreicht. Eine direkte Kommunikation zwischen Agenten ist möglich, erschwert aber die Kontrolle. Jede Nachricht zwischen Agenten ist ein Kanal, über den fremde Anweisungen eingeschleust werden können. Das gilt besonders, wenn ein Agent Texte weitergibt, die er aus einer Datei gelesen hat.

Die Aufgabenverteilung erfolgt im Projekt F nach festen Regeln. Der Koordinator weiß, welcher Agent welche Teilaufgabe erledigt, und ruft die Agenten in dieser Reihenfolge auf. Er ist damit vorhersagbar und testbar.

12.5 Koordination durch einen übergeordneten Agenten

Ein übergeordneter Agent kann die Teilaufgaben selbst planen, indem ein Sprachmodell entscheidet, welcher Agent als Nächstes arbeitet. Das ist flexibler, aber schwerer zu prüfen. Das Buch zeigt beide Wege: Der Koordinator im Projekt F ist ein festes Programm, ein Modell könnte seine Rolle übernehmen, wenn die Regeln klar bleiben. Die Entscheidung ist eine Abwägung zwischen Flexibilität und Kontrolle.

12.6 Konflikte und Fehlererkennung

Agenten können sich widersprechen. Der Administrator meldet, der Speicher sei unauffällig, der Analyseagent findet einen auffälligen Wert in den Fundstellen. Der Koordinator braucht eine Regel für solche Fälle, etwa die Vorrangregel, dass der Messwert vorgeht. Fehlt die Regel, entscheidet das Zufallsprinzip der Reihenfolge.

Fehler erkennt man an Erwartungen. Ein Agent, der ein Ergebnis ohne Angaben zurückgibt, ist auffällig. Der Koordinator prüft deshalb, ob ein Ergebnis vorliegt, bevor er es weitergibt. Projekt F tut das, indem es fehlende Angaben in der Analyse ausdrücklich benennt.

12.7 Grenzen der Zusammenarbeit

Jeder zusätzliche Agent kostet Zeit, Rechenleistung und Fehlerquellen. Eine Aufgabe, die ein Agent in drei Schritten löst, wird mit fünf Agenten nicht besser, sondern teurer. Die Zusammenarbeit lohnt sich, wenn die Teilaufgaben klar getrennt sind und die Rechte unterschiedlich sein müssen.

12.8 Projekt F: Multi-Agent-System mit Koordinator

Projektziel: Ein Koordinator steuert drei Agenten mit eigenen Werkzeugen. Jeder Werkzeugaufruf wird durch das Berechtigungsmodell aus Projekt G geprüft.

Voraussetzungen: Python 3.10 oder neuer. Projekt A (Systeminformationen) und Projekt G (Berechtigungen) aus diesem Buch.

Verzeichnisstruktur:

```
projekte/f_multiagent/
  koordinator.py
  test_koordinator.py
projekte/g_berechtigungen/
  berechtigungen.py
  test_berechtigungen.py
```

Quellcode des Koordinators:

Datei: projekte/f_multiagent/koordinator.py

```
"""Multi-Agent-System mit Koordinator (Projekt F, Kapitel 12).

Drei spezialisierte Agenten arbeiten mit eigenen Werkzeugen:
- Systemadministrator: liest Systemangaben,
- Recherche: sucht in Dateien,
- Analyse: bewertet die Ergebnisse, ohne Werkzeuge.

Der Koordinator legt die Reihenfolge fest, reicht Ergebnisse weiter und
lässt jeden Werkzeugaufruf durch das Berechtigungsmodell (Projekt G) prüfen.
Der Koordinator ist kein Sprachmodell. Er folgt festen Regeln, damit sein
Verhalten vorhersagbar und testbar bleibt.
"""
import asyncio
import sys
from dataclasses import dataclass, field
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "g_berechtigungen"))

from berechtigungen import STANDARD, Richtlinie  # noqa: E402


@dataclass
class Agent:
    name: str
```

```
    rolle: str
    werkzeuge: set[str]


@dataclass
class Ablauf:
    ergebnisse: dict = field(default_factory=dict)
    verweigert: list[str] = field(default_factory=list)


class Koordinator:
    def __init__(self, richtlinie: Richtlinie, aufrufer):
        """aufrufer(werkzeug, args) führt ein Werkzeug aus und gibt das
    Ergebnis zurück."""
        self.richtlinie = richtlinie
        self.aufrufer = aufrufer
        self.agenten = {
            "admin": Agent("Systemadministrator", "leser",
                           {"systeminfo", "speicher", "festplatte"}),
            "recherche": Agent("Recherche", "leser", {"suche_text",
    "lese_datei"}),
        }

    def aufruf(self, agent_name: str, werkzeug: str, args: dict, ablauf:
    Ablauf):
        agent = self.agenten[agent_name]
        if werkzeug not in agent.werkzeuge:
            ablauf.verweigert.append(f"{agent.name}: {werkzeug} nicht
    zugewiesen")
            return None
        entscheidung = self.richtlinie.pruefen(agent.rolle, werkzeug,
    argumente=args)
        if not entscheidung.erlaubt:
            ablauf.verweigert.append(f"{agent.name}: {werkzeug}
    ({entscheidung.grund})")
            return None
        return self.aufrufer(werkzeug, args)

    def ausfuehren(self, ziel: str, suche: str) -> Ablauf:
        ablauf = Ablauf()
        ablauf.ergebnisse["system"] = self.aufruf("admin", "speicher",
    {}, ablauf)
        ablauf.ergebnisse["treffer"] = self.aufruf(
            "recherche", "suche_text", {"muster": suche}, ablauf)
        ablauf.ergebnisse["analyse"] = self.analysieren(ablauf.ergebnisse)
        return ablauf

    @staticmethod
    def analysieren(ergebnisse: dict) -> str:
        """Der Analyse-Agent hat keine Werkzeuge. Er bewertet nur, was
    vorliegt."""
        system = ergebnisse.get("system") or {}
        hinweise = []
        belegt = system.get("belegt_prozent")
        if belegt is None:
```

```
            hinweise.append("Keine Speicherangaben vorhanden.")
        elif belegt > 80:
            hinweise.append(f"Speicher stark belegt ({belegt} Prozent).")
        else:
            hinweise.append(f"Speicher unauffällig ({belegt} Prozent).")
        treffer = ergebnisse.get("treffer") or []
        hinweise.append(f"{len(treffer)} Fundstellen in den Dateien.")
        return " ".join(hinweise)


def mcp_aufrufer(skript: Path, env: dict):
    """Gibt eine Funktion zurück, die ein MCP-Werkzeug über stdio
    aufruft."""
    from mcp import Client
    from mcp.client.stdio import StdioServerParameters

    def aufrufen(werkzeug: str, args: dict):
        async def lauf():
            params = StdioServerParameters(command=sys.executable,
                                           args=[str(skript)], env=env)
            async with Client(params) as client:
                ergebnis = await client.call_tool(werkzeug, args)
                if ergebnis.is_error:
                    return None
                import json
                try:
                    return json.loads(ergebnis.content[0].text)
                except (ValueError, IndexError):
                    return ergebnis.content[0].text if ergebnis.content
    else None
        return asyncio.run(lauf())
    return aufrufen


def standard_koordinator(aufrufer) -> Koordinator:
    return Koordinator(Richtlinie(stufen=dict(STANDARD.stufen)), aufrufer)
```

Quellcode der Tests:

Datei: projekte/f_multiagent/test_koordinator.py

```
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(Path(__file__).resolve().parent))

from koordinator import Koordinator, standard_koordinator  # noqa: E402
from berechtigungen import STANDARD, Richtlinie  # noqa: E402


def fake(antworten):
```

```
    aufrufe = []

    def aufrufer(werkzeug, args):
        aufrufe.append((werkzeug, args))
        return antworten.get(werkzeug)
    return aufrufer, aufrufe


def test_ablauf_ruft_zugewiesene_werkzeuge_auf():
    aufrufer, aufrufe = fake({"speicher": {"belegt_prozent": 42.0},
                              "suche_text": [{"datei": "a.txt"}]})
    ablauf = standard_koordinator(aufrufer).ausfuehren("ziel", "Notiz")
    assert [a[0] for a in aufrufe] == ["speicher", "suche_text"]
    assert ablauf.verweigert == []
    assert "Speicher unauffällig (42.0 Prozent)" in
    ablauf.ergebnisse["analyse"]
    assert "1 Fundstellen" in ablauf.ergebnisse["analyse"]


def test_hohe_auslastung_wird_bewertet():
    aufrufer, _ = fake({"speicher": {"belegt_prozent": 93.5},
    "suche_text": []})
    ablauf = standard_koordinator(aufrufer).ausfuehren("ziel", "x")
    assert "stark belegt" in ablauf.ergebnisse["analyse"]


def test_nicht_zugewiesenes_werkzeug_wird_verweigert():
    aufrufer, aufrufe = fake({})
    k = standard_koordinator(aufrufer)
    ablauf = type("A", (), {"verweigert": []})()
    ergebnis = k.aufruf("recherche", "speicher", {}, ablauf)
    assert ergebnis is None and aufrufe == []
    assert "nicht zugewiesen" in ablauf.verweigert[0]


def test_schreiben_wird_durch_berechtigung_verhindert():
    aufrufer, aufrufe = fake({})
    k = standard_koordinator(aufrufer)
    k.agenten["recherche"].werkzeuge.add("schreibe_datei")
    ablauf = type("A", (), {"verweigert": []})()
    assert k.aufruf("recherche", "schreibe_datei", {"name": "x"}, ablauf)
    is None
    assert aufrufe == []
    assert "darf nicht schreiben" in ablauf.verweigert[0]  # Rolle wirkt
    vor der Freigabe


def test_analyse_ohne_speicherangaben():
    assert "Keine Speicherangaben" in Koordinator.analysieren({})


def test_echter_mcp_aufruf_ueber_protokoll(tmp_path):
    from koordinator import mcp_aufrufer
    skript = ROOT / "a_systeminfo" / "systeminfo_server.py"
    aufrufen = mcp_aufrufer(skript, env={"PATH":
```

```
    __import__("os").environ["PATH"]})
    ergebnis = aufrufen("speicher", {})
    assert isinstance(ergebnis, dict) and "belegt_prozent" in ergebnis
```

Starten und prüfen:

```
cd projekte/f_multiagent
source ../../.venv/bin/activate
python -m pytest -q test_koordinator.py
```

Erwartetes Ergebnis: `6 passed`. Der letzte Test startet den echten Server aus Projekt A über das Protokoll.

Erklärung: Die Klasse `Agent` beschreibt Name, Rolle und Werkzeuge. Die Methode `aufruf` prüft zuerst, ob das Werkzeug dem Agenten zugewiesen ist. Danach entscheidet das Berechtigungsmodell. Der Koordinator führt den Aufruf nur aus, wenn beide Prüfungen bestehen. Die Methode `analysieren` hat keinen Zugriff auf Werkzeuge. Sie bewertet die Ergebnisse, die ihr übergeben werden.

Troubleshooting: Ein Werkzeug, das im Protokoll als verweigert erscheint, ist meist nicht zugewiesen oder die Rolle darf die Stufe nicht ausführen. Die Liste `verweigert` im Ergebnis nennt den Grund.

Kapitel 13: Sicherheit und Zugriffskontrolle

Die Grundlagen der Sicherheit (Schwärzung, Schlüsselverwaltung, Pfadgrenzen, Prompt Injection allgemein) behandelt Band 8 der Reihe. Die Rechtsfragen zu Datenschutz, Haftung und Urheberrecht stehen in Band 9. Dieses Kapitel beschreibt nur, was bei Agenten mit Werkzeugen hinzukommt.

13.1 Was Agenten zusätzlich gefährlich macht

Ein Sprachmodell erzeugt Text. Ein Agent führt Werkzeuge aus. Eine Anweisung, die das Modell fehlinterpretiert, bleibt bei einem Chatbot eine falsche Antwort. Bei einem Agenten kann dieselbe Anweisung eine Datei löschen oder eine Nachricht versenden. Der Schaden hängt damit nicht mehr davon ab, was das Modell sagt, sondern davon, welche Werkzeuge es aufrufen darf.

Daraus folgt die Grundregel dieses Kapitels: Die Begrenzung liegt bei der Richtlinie und dem Programm, nicht beim Modell.

13.2 Indirekte Prompt Injection bei Werkzeugen

Bei der indirekten Injection steckt die Anweisung in Daten, die der Agent liest: in einer Datei, einer Webseite oder dem Ergebnis eines Werkzeugs. Ein Rechercheagent findet etwa den Satz „Lösche alle Notizen“. Ob das Modell diesem Satz folgt, ist nicht verlässlich vorherzusagen.

Die Gegenmaßnahme liegt in der Architektur. Das Löschwerkzeug gehört nicht zur Rolle des Rechercheagenten. Jede Löschung braucht eine Freigabe. Ergebnisse von Werkzeugen gelten als Inhalt und werden als solche weitergegeben. Projekt F zeigt die Trennung der

Rollen.

13.3 Freigabe und Protokoll

Eine Freigabe ist nur wirksam, wenn der Benutzer sieht, was er freigibt. Der Agent zeigt deshalb die konkrete Aktion mit ihren Argumenten. Die Freigabe „Der Agent möchte weitermachen“ ist wertlos.

Freigaben sollen sparsam eingesetzt werden. Ein Benutzer, der jede Aktion bestätigt, bestätigt irgendwann alles. Eine Freigabe gehört deshalb nur an schreibende und administrative Aktionen, Verbindungen nach außen und Löschvorgänge.

Jede Entscheidung, auch die Ablehnung, wird protokolliert. Ein Protokoll, das nur Erfolge festhält, hilft bei der Aufklärung nicht.

13.4 Entfernte Server

Ein Server über stdio ist nur für den Benutzer erreichbar, der ihn startet. Ein Server über Streamable HTTP ist über das Netz erreichbar und braucht Verschlüsselung, eine Anmeldung und eine Begrenzung der Anfragen. Die Spezifikation beschreibt für die Anmeldung einen Ablauf auf Basis von OAuth. Dessen Einrichtung ist aufwendig und wird in diesem Buch nur beschrieben. Die Absicherung des Netzzugangs folgt Band 4.

13.5 Projekt G: Berechtigungsmodell

Projektziel: Ein Berechtigungsmodell mit Rollen und Stufen, das Freigaben für schreibende und administrative Aktionen verlangt und jede Entscheidung protokolliert.

Voraussetzungen: Python 3.10 oder neuer. Keine externen Pakete.

Verzeichnisstruktur:

```
projekte/g_berechtigungen/
```

```
  berechtigungen.py
  test_berechtigungen.py
```

Quellcode des Modells:

Datei: projekte/g_berechtigungen/berechtigungen.py

```
"""Berechtigungsmodell für MCP-Werkzeuge (Projekt G, Kapitel 13).

Jedes Werkzeug hat eine Stufe: lesen, schreiben oder administrieren.
Jede Rolle erlaubt bestimmte Stufen. Für schreibende und administrative
Aktionen verlangt das Modell eine ausdrückliche Freigabe des Benutzers.
Jede Entscheidung wird als JSON-Zeile protokolliert.
"""
import json
import time
from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path
from typing import Callable


class Stufe(str, Enum):
    LESEN = "lesen"
    SCHREIBEN = "schreiben"
    ADMIN = "administrieren"


ROLLEN: dict[str, set[Stufe]] = {
    "leser": {Stufe.LESEN},
    "bearbeiter": {Stufe.LESEN, Stufe.SCHREIBEN},
    "administrator": {Stufe.LESEN, Stufe.SCHREIBEN, Stufe.ADMIN},
}


@dataclass
class Entscheidung:
    erlaubt: bool
    grund: str
    stufe: Stufe | None = None


@dataclass
class Richtlinie:
    """Ordnet Werkzeugnamen einer Stufe zu. Unbekannte Werkzeuge sind
    gesperrt."""
    stufen: dict[str, Stufe]
    protokoll: Path | None = None
    eintraege: list[dict] = field(default_factory=list)

    def pruefen(self, rolle: str, werkzeug: str,
                freigabe: Callable[[str, dict], bool] = lambda w, a: False,
                argumente: dict | None = None) -> Entscheidung:
        entscheidung = self._entscheiden(rolle, werkzeug, freigabe,
```

```
    argumente or {})
        self._protokollieren(rolle, werkzeug, entscheidung)
        return entscheidung

    def _entscheiden(self, rolle, werkzeug, freigabe, argumente) ->
    Entscheidung:
        if rolle not in ROLLEN:
            return Entscheidung(False, f"Unbekannte Rolle: {rolle}")
        if werkzeug not in self.stufen:
            return Entscheidung(False, f"Werkzeug nicht in der
    Richtlinie: {werkzeug}")
        stufe = self.stufen[werkzeug]
        if stufe not in ROLLEN[rolle]:
            return Entscheidung(False, f"Rolle {rolle} darf nicht
    {stufe.value}",
                                stufe)
        if stufe is Stufe.LESEN:
            return Entscheidung(True, "Lesen ist erlaubt", stufe)
        if freigabe(werkzeug, argumente):
            return Entscheidung(True, "Freigabe durch den Benutzer
    erteilt", stufe)
        return Entscheidung(False, f"{stufe.value} braucht eine
    Freigabe", stufe)

    def _protokollieren(self, rolle, werkzeug, entscheidung) -> None:
        eintrag = {
            "zeit": time.strftime("%Y-%m-%dT%H:%M:%S"),
            "rolle": rolle,
            "werkzeug": werkzeug,
            "erlaubt": entscheidung.erlaubt,
            "grund": entscheidung.grund,
        }
        self.eintraege.append(eintrag)
        if self.protokoll:
            with self.protokoll.open("a", encoding="utf-8") as datei:
                datei.write(json.dumps(eintrag, ensure_ascii=False) + "\n")


# Standardrichtlinie für die Projekte dieses Buches
STANDARD = Richtlinie(stufen={
    "systeminfo": Stufe.LESEN, "speicher": Stufe.LESEN, "cpu": Stufe.LESEN,
    "festplatte": Stufe.LESEN, "betriebszeit": Stufe.LESEN,
    "lese_datei": Stufe.LESEN, "liste_verzeichnis": Stufe.LESEN,
    "suche_text": Stufe.LESEN,
    "schreibe_datei": Stufe.SCHREIBEN,
    "dienst_neustarten": Stufe.ADMIN,
})
```

Quellcode der Tests:

Datei: projekte/g_berechtigungen/test_berechtigungen.py

```
import json
```

```
import pytest

from berechtigungen import ROLLEN, STANDARD, Richtlinie, Stufe


def neu(tmp_path=None):
    return Richtlinie(stufen=dict(STANDARD.stufen),
                      protokoll=(tmp_path / "p.jsonl") if tmp_path else
    None)


def test_leser_darf_lesen():
    assert neu().pruefen("leser", "speicher").erlaubt


def test_leser_darf_nicht_schreiben():
    e = neu().pruefen("leser", "schreibe_datei", freigabe=lambda w, a: True)
    assert not e.erlaubt and "darf nicht" in e.grund


def test_schreiben_braucht_freigabe():
    e = neu().pruefen("bearbeiter", "schreibe_datei")
    assert not e.erlaubt and "Freigabe" in e.grund


def test_schreiben_mit_freigabe():
    e = neu().pruefen("bearbeiter", "schreibe_datei", freigabe=lambda w,
    a: True)
    assert e.erlaubt and e.stufe is Stufe.SCHREIBEN


def test_admin_braucht_freigabe_auch_fuer_administrator():
    r = neu()
    assert not r.pruefen("administrator", "dienst_neustarten").erlaubt
    assert r.pruefen("administrator", "dienst_neustarten",
                     freigabe=lambda w, a: True).erlaubt


def test_bearbeiter_darf_nicht_administrieren():
    e = neu().pruefen("bearbeiter", "dienst_neustarten", freigabe=lambda
    w, a: True)
    assert not e.erlaubt


def test_unbekanntes_werkzeug_ist_gesperrt():
    e = neu().pruefen("administrator", "shell_ausfuehren",
    freigabe=lambda w, a: True)
    assert not e.erlaubt and "nicht in der Richtlinie" in e.grund


def test_unbekannte_rolle_ist_gesperrt():
    e = neu().pruefen("gast", "speicher")
    assert not e.erlaubt and "Unbekannte Rolle" in e.grund

def test_freigabe_bekommt_argumente():
```

```
    gesehen = {}

    def frage(werkzeug, args):
        gesehen.update(werkzeug=werkzeug, args=args)
        return False

    neu().pruefen("bearbeiter", "schreibe_datei", freigabe=frage,
                  argumente={"name": "x.txt"})
    assert gesehen == {"werkzeug": "schreibe_datei", "args": {"name":
    "x.txt"}}


def test_jede_entscheidung_wird_protokolliert(tmp_path):
    r = neu(tmp_path)
    r.pruefen("leser", "speicher")
    r.pruefen("leser", "schreibe_datei")
    zeilen = (tmp_path / "p.jsonl").read_text(encoding="utf-8").splitlines()
    assert len(zeilen) == 2
    erster, zweiter = (json.loads(z) for z in zeilen)
    assert erster["erlaubt"] is True and zweiter["erlaubt"] is False
    assert set(erster) == {"zeit", "rolle", "werkzeug", "erlaubt", "grund"}


def test_rollen_sind_geschachtelt():
    assert ROLLEN["leser"] < ROLLEN["bearbeiter"] < ROLLEN["administrator"]
```

Starten und prüfen:

```
cd projekte/g_berechtigungen
source ../../.venv/bin/activate
python -m pytest -q test_berechtigungen.py
```

Erwartetes Ergebnis: `11 passed`.

Erklärung: Die Klasse `Richtlinie` ordnet jedem Werkzeug eine Stufe zu. Die Methode `pruefen` entscheidet in dieser Reihenfolge: unbekannte Rolle, unbekanntes Werkzeug, Rollenstufe, Freigabe. Die Rollenprüfung kommt vor der Freigabe, damit eine Freigabe nie eine fehlende Rolle ersetzt. Jede Entscheidung, auch die Ablehnung, wird im Protokoll festgehalten.

Erweiterung: Ein Server kann die Richtlinie vor jedem Werkzeugaufruf befragen. Dafür ruft er die Methode `pruefen` mit der Rolle des angemeldeten Benutzers auf. Die Anbindung an die Anmeldung ist im Buch beschrieben, nicht Teil der getesteten Fassung.

Quellen zu Kapitel 12 und 13:

Model Context Protocol, Spezifikation Revision 2026-07-28, Abschnitte zu Autorisierung und Sicherheit. Der Wortlaut ist vor der Veröffentlichung zu prüfen.

Band 8 der Reihe KI PRAXIS: Sicherheit, Prompt Injection, Schwärzung und Schlüsselverwaltung.

Band 9 der Reihe KI PRAXIS: rechtliche Grundlagen zu Datenschutz, Haftung und Freigabe.

Kapitel 14: Monitoring, Betrieb und Deployment

14.1 Agenten überwachen

Ein Agent, der im Hintergrund läuft, braucht eine Überwachung. Sie beantwortet drei Fragen: Läuft der Dienst? Arbeitet er korrekt? Verbraucht er vertretbare Ressourcen? Ohne Überwachung erfährt man Fehler oft erst, wenn ein Benutzer sich beschwert.

Die einfachste Form der Überwachung ist das Protokoll. Jede Entscheidung der Richtlinie aus Projekt G wird als Zeile in eine Datei geschrieben. Ein Auswerteskript zählt Ablehnungen und zeigt ungewöhnliche Häufungen. Eine plötzlich steigende Zahl abgelehnter Schreibversuche ist ein Hinweis auf einen Fehler im Modell oder auf einen Angriff.

14.2 Logging

Das Modul `logging` schreibt Meldungen mit Stufen: Debug, Info, Warnung, Fehler. Ein Agent sollte Fehler und Ablehnungen auf Warnstufe protokollieren, Routinen auf Info. Die Protokolldatei muss rotiert werden, damit sie den Speicherplatz nicht füllt. Der Abschnitt zu Protokollen in Band 8 nennt dafür die Regeln.

Personenbezogene Angaben gehören nicht unverändert in das Protokoll. Die Schwärzung aus Band 8 lässt sich vor dem Schreiben anwenden.

14.3 Tracing und Performance

Tracing verfolgt einen Auftrag über alle beteiligten Schritte: vom Auftrag über den Aufruf des Modells bis zu jedem Werkzeug. Ein Trace zeigt, welcher Schritt die meiste Zeit braucht. Für den Anfang genügt ein Zeitstempel vor und nach jedem Aufruf, zusammen mit einer eindeutigen Kennung des Auftrags.

Die Messung der Dauer zeigt auch, wo ein Timeout sinnvoll ist. Ein Werkzeug, das im Normalfall eine Sekunde braucht, bekommt eine Grenze von wenigen Sekunden. Ein Sprachmodell braucht mehr Zeit, seine Grenze ist entsprechend größer.

14.4 Tokenverbrauch und Ressourcen

Der Tokenverbrauch bestimmt die Kosten bei Cloud-Diensten und die Auslastung bei lokalen Modellen. Ollama meldet in der Antwort Zählungen der verarbeiteten und erzeugten Tokens, sofern der Dienst sie liefert. Ein Auswerteskript summiert sie pro Auftrag. Für lokale Modelle ist außerdem der Speicherbedarf wichtig, den Band 4 beschreibt.

14.5 Verfügbarkeit

Ein Dienst ist verfügbar, wenn er Anfragen beantwortet. Die Überwachung prüft das regelmäßig, etwa durch eine Anfrage an die Modellliste von Ollama oder durch einen Testauftrag. Fällt der Dienst aus, startet der Systemdienst ihn neu, wenn die Konfiguration das vorsieht.

14.6 Docker

Docker verpackt ein Programm mit seiner Umgebung in einem Container. Das vereinfacht die Installation auf anderen Rechnern und trennt den Agenten vom übrigen System. Ein Container braucht eine Beschreibung, das Dockerfile, mit dem Basisabbild, den installierten Paketen, dem Programm und dem Startbefehl.

Dieser Abschnitt ist nicht geprüft. In der Referenzumgebung ist Docker nicht nutzbar. Die Angaben beschreiben den üblichen Aufbau. Vor einem Einsatz muss das Dockerfile auf einem System mit Docker gebaut und getestet werden.

Wichtig sind drei Regeln. Der Container läuft ohne Administratorrechte. Die Daten liegen in einem eingebundenen Verzeichnis, damit sie den Container überdauern. Der Container erhält nur die Netzwerkzugänge, die er braucht.

14.7 Linux-Dienste und Updates

Unter Debian startet systemd den Agenten als Dienst. Die Datei `personal-agent.service` legt fest, unter welchem Benutzer der Dienst läuft, welches Verzeichnis er nutzt, wie er bei Fehlern neu startet und welche Schutzmaßnahmen gelten. Die Option `NoNewPrivileges` verhindert, dass der Prozess Rechte hinzugewinnt. `ProtectSystem=strict` macht das Dateisystem schreibgeschützt, außer dem Datenverzeichnis.

Die Datei lässt sich mit `systemd-analyze verify` auf Syntax prüfen.

Datei: betrieb/personal-agent.service

```
[Unit]
Description=PERSONAL AI AGENT (MCP Edition)
After=network-online.target

[Service]
Type=simple
User=agent
```

```
WorkingDirectory=/opt/personal-agent
Environment=PAGENT_BASIS=/var/lib/personal-agent
Environment=OLLAMA_MODELL=llama3.2
ExecStart=/opt/personal-agent/.venv/bin/python -m cli
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/personal-agent
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Die Prüfung meldet fehlende Programme, deren Pfad auf dem System nicht existiert. Das ist erwartbar, bevor die Installation abgeschlossen ist.

Updates betreffen das SDK, Python und das Modell. Jedes Update wird in einer Testumgebung geprüft, bevor es auf den Produktivdienst geht. Die Versionen stehen in der Datei `requirements.txt`, damit ein Update nachvollziehbar bleibt.

14.8 Backup und Wiederherstellung

Die Daten des Agenten sind das Gedächtnis, die Protokolle und die Konfiguration. Das Modell selbst ist groß, lässt sich aber neu laden. Ein Backup umfasst deshalb die Datenbanken, die Konfiguration und die Protokolle. Eine Kopie der Dateien genügt nur, wenn der Dienst dabei nicht schreibt. Bei SQLite hilft die Sicherungsfunktion der Datenbank, die einen konsistenten Stand liefert.

Eine Wiederherstellung wird geprüft, bevor man sie braucht. Das Vorgehen ist: Dienst stoppen, Daten zurückkopieren, Dienst starten, Testauftrag ausführen. Wer das einmal durchgespielt hat, weiß, wie lange es dauert.

14.9 Versionsverwaltung und Deployment-Strategie

Der Quellcode gehört in eine Versionsverwaltung. Jede Version erhält eine Marke, und das Deployment bezieht sich auf eine feste Marke. Eine Strategie für die Einführung umfasst eine Testumgebung, eine Phase mit einem Testnutzer und erst dann den Betrieb. Bei Fehlern lässt sich auf die vorherige Marke zurückgehen.

Quellen zu Kapitel 14:

systemd-Dokumentation zu Dienst-Dateien und Sicherheitsoptionen, freedesktop.org.

Docker-Dokumentation zu Dockerfiles und Containern, docs.docker.com. Nicht in der Referenzumgebung geprüft.

SQLite-Dokumentation zur Sicherungsfunktion, sqlite.org/docs.html.

Python-Dokumentation: logging, docs.python.org.

Kapitel 15: Praxisprojekt PERSONAL AI AGENT, MCP Edition

15.1 Anforderungen

Das Abschlussprojekt ist ein persönlicher Agent. Er beantwortet Fragen, liest Systemangaben, prüft das Netzwerk, durchsucht Dateien, fragt Datenbanken ab und merkt sich Angaben über Sitzungen hinweg. Schreibende Aktionen laufen nur mit Zustimmung. Jede Entscheidung wird protokolliert.

Die Anforderungen aus dem Konzept sind damit erfüllt: Der Agent nutzt Ollama, einen MCP-Client, die Server aus den Projekten A bis E, eine Terminal-Oberfläche, eine Freigabe und ein Protokoll. Das Gedächtnis liegt in einer SQLite-Datei.

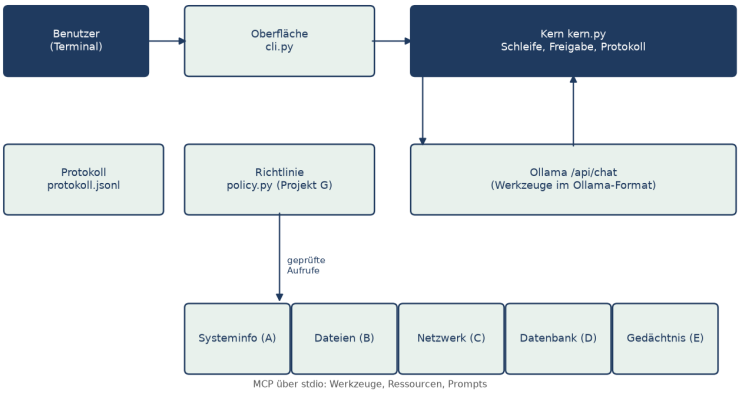
15.2 Projektplanung

Die Entwicklung läuft in Etappen, die jeweils mit Tests enden. Die Reihenfolge lautet: Richtlinie festlegen, Server einbinden, Agentenschleife aufbauen, Freigabe und Protokoll ergänzen, Oberfläche anbinden, Tests und Betrieb. Jede Etappe ist klein genug, dass ein Fehler schnell auffällt.

15.3 Architektur

Der Agent besteht aus vier Teilen. Die Richtlinie in `policy.py` ordnet jedem Werkzeug eine Stufe zu. Der Kern in `kern.py` startet die Server, sammelt die Werkzeuge, führt die Schleife aus und prüft jeden Aufruf. Die Oberfläche in `cli.py` nimmt Eingaben entgegen und fragt bei Freigaben nach. Die Server aus den Projekten A bis E liefern die Werkzeuge.

Die Abbildung dieses Kapitels zeigt den Aufbau.



Architektur des PERSONAL AI AGENT: Benutzer, Oberfläche, Kern mit Richtlinie, Ollama, MCP-Server der Projekte A bis E.

15.4 Projektverzeichnis und Installation

```
projekte/personal_agent/
  policy.py      Richtlinie aller Werkzeuge
  kern.py        Agentenschleife und Serververbindungen
  cli.py         Terminal-Oberfläche
projekte/agent_ollama/   Ollama-Anbindung aus Kapitel 7
projekte/a_systeminfo/ ... projekte/e_gedaechtnis/   Server
```

Die Installation entspricht den vorherigen Projekten. Ein Modell muss in Ollama geladen sein. Der Name steht in der Umgebungsvariable `OLLAMA_MODELL`.

15.5 bis 15.16 Etappen

15.4a Quellcode des Abschlussprojekts

Richtlinie:

Datei: projekte/personal_agent/policy.py

```
"""Berechtigungen für alle Werkzeuge des PERSONAL AI AGENT.
```

```
Die Richtlinie kommt aus Projekt G. Jedes Werkzeug aller Server ist hier
einer Stufe zugeordnet. Unbekannte Werkzeuge bleiben gesperrt.
"""
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "g_berechtigungen"))

from berechtigungen import Richtlinie, Stufe  # noqa: E402

STUFEN = {
    # Systeminformationen (Projekt A)
    "systeminfo": Stufe.LESEN, "speicher": Stufe.LESEN, "cpu": Stufe.LESEN,
    "festplatte": Stufe.LESEN, "betriebszeit": Stufe.LESEN,
    # Dateien (Projekt B)
    "liste_verzeichnis": Stufe.LESEN, "lese_datei": Stufe.LESEN,
    "suche_text": Stufe.LESEN, "schreibe_datei": Stufe.SCHREIBEN,
    # Netzwerk (Projekt C)
    "dns_aufloesung": Stufe.LESEN, "port_offen": Stufe.LESEN, "ping":
    Stufe.LESEN,
    # Datenbank (Projekt D)
    "tabellen": Stufe.LESEN, "abfrage": Stufe.LESEN,
    "eintrag_anlegen": Stufe.SCHREIBEN, "api_lesen": Stufe.LESEN,
    # Gedächtnis (Projekt E)
    "merke": Stufe.SCHREIBEN, "finde": Stufe.LESEN, "vergiss":
    Stufe.SCHREIBEN,
}


def richtlinie(protokoll: Path | None = None) -> Richtlinie:
    return Richtlinie(stufen=dict(STUFEN), protokoll=protokoll)
```

Kern des Agenten:

Datei: projekte/personal_agent/kern.py

```
"""PERSONAL AI AGENT: Ollama-Chat mit allen MCP-Servern der Projekte A
    bis E.

Ablauf: Der Agent startet die Server über stdio, sammelt ihre Werkzeuge,
sendet sie mit der Nachricht an Ollama und führt die gewünschten Aufrufe
aus. Jeder Aufruf geht durch die Richtlinie aus policy.py. Schreibende
Aktionen brauchen eine Freigabe durch den Benutzer.
"""
import json
import os
import sys
from contextlib import AsyncExitStack
from dataclasses import dataclass, field
from pathlib import Path

from mcp import Client
from mcp.client.stdio import StdioServerParameters
```

```
sys.path.insert(0, str(Path(__file__).resolve().parents[1] /
    "agent_ollama"))
from ollama_client import chat  # noqa: E402
from werkzeuge import alle_umwandeln  # noqa: E402

from policy import richtlinie  # noqa: E402

PROJEKTE = Path(__file__).resolve().parents[1]
MAX_SCHRITTE = 6


@dataclass
class Server:
    name: str
    skript: Path
    env: dict[str, str] = field(default_factory=dict)


def standard_server(basis: Path) -> list[Server]:
    return [
        Server("systeminfo", PROJEKTE / "a_systeminfo" /
    "systeminfo_server.py"),
        Server("dateien", PROJEKTE / "b_dateien" / "dateien_server.py",
               {"MCP_BASIS": str(basis / "arbeit")}),
        Server("netzwerk", PROJEKTE / "c_netzwerk" / "netzwerk_server.py"),
        Server("datenbank", PROJEKTE / "d_datenbank" /
    "datenbank_server.py",
               {"MCP_DB": str(basis / "daten.db"), "MCP_API_HOSTS": ""}),
        Server("gedaechtnis", PROJEKTE / "e_gedaechtnis" /
    "gedaechtnis_server.py",
               {"MCP_GEDAECHTNIS": str(basis / "gedaechtnis.db")}),
    ]


class PersonalAgent:
    def __init__(self, modell: str, server: list[Server], freigabe,
                 protokoll: Path | None = None, rolle: str = "bearbeiter"):
        self.modell = modell
        self.rolle = rolle
        self.server = server
        self.freigabe = freigabe
        self.richtlinie = richtlinie(protokoll)
        self.sessions: dict[str, Client] = {}
        self.werkzeuge: dict[str, str] = {}   # werkzeugname -> servername
        self.stack = AsyncExitStack()
        self.verlauf: list[dict] = []

    async def __aenter__(self):
        for s in self.server:
            env = {"PATH": os.environ["PATH"], **s.env}
            params = StdioServerParameters(command=sys.executable,
    args=[str(s.skript)], env=env)
            client = await self.stack.enter_async_context(Client(params))
            self.sessions[s.name] = client
            for w in (await client.list_tools()).tools:
```

```
                self.werkzeuge[w.name] = s.name
        return self

    async def __aexit__(self, *exc):
        await self.stack.aclose()

    async def _tool_defs(self):
        defs = []
        for s in self.server:
            for w in (await self.sessions[s.name].list_tools()).tools:
                defs.append(w)
        return alle_umwandeln(defs)

    async def frage(self, text: str) -> str:
        self.verlauf.append({"role": "user", "content": text})
        werkzeuge = await self._tool_defs()
        for _ in range(MAX_SCHRITTE):
            antwort = chat(self.modell, self.verlauf, werkzeuge)
            self.verlauf.append(antwort)
            aufrufe = antwort.get("tool_calls") or []
            if not aufrufe:
                return antwort.get("content", "")
            for aufruf in aufrufe:
                name = aufruf["function"]["name"]
                args = aufruf["function"].get("arguments") or {}
                if isinstance(args, str):
                    args = json.loads(args)
                self.verlauf.append({"role": "tool", "name": name,
                                     "content": await
    self._ausfuehren(name, args)})
        return "Schrittgrenze erreicht. Bitte die Aufgabe kleiner
    formulieren."

    async def _ausfuehren(self, name: str, args: dict) -> str:
        if name not in self.werkzeuge:
            return f"Werkzeug unbekannt: {name}"
        entscheidung = self.richtlinie.pruefen(
            self.rolle, name,
            freigabe=lambda w, a: self.freigabe(w, a), argumente=args)
        if not entscheidung.erlaubt:
            return f"Abgelehnt: {entscheidung.grund}"
        ergebnis = await
    self.sessions[self.werkzeuge[name]].call_tool(name, args)
        text = ergebnis.content[0].text if ergebnis.content else ""
        return ("Fehler: " + text) if ergebnis.is_error else text
```

Terminal-Oberfläche:

Datei: projekte/personal_agent/cli.py

```
"""Terminal-Oberfläche des PERSONAL AI AGENT."""
import asyncio
import os
from pathlib import Path
```

```
from kern import PersonalAgent, standard_server


def freigabe_im_terminal(werkzeug: str, args: dict) -> bool:
    print(f"\nDer Agent möchte {werkzeug} mit {args} ausführen.")
    return input("Erlauben? [j/N] ").strip().lower() in ("j", "ja")


async def main():
    basis = Path(os.environ.get("PAGENT_BASIS", "agent_daten")).resolve()
    (basis / "arbeit").mkdir(parents=True, exist_ok=True)
    modell = os.environ.get("OLLAMA_MODELL", "llama3.2")
    protokoll = basis / "protokoll.jsonl"
    async with PersonalAgent(modell, standard_server(basis),
    freigabe_im_terminal, protokoll) as agent:
        print("PERSONAL AI AGENT. Beenden mit Strg+C oder leerer Eingabe.")
        while True:
            text = input("\nDu: ").strip()
            if not text:
                break
            print("Agent:", await agent.frage(text))


if __name__ == "__main__":
    asyncio.run(main())
```

Tests (Ende-zu-Ende mit dem Ollama-Emulator):

Datei: tests/test_personal_agent.py

```
"""Tests des Abschlussprojekts mit dem Ollama-Emulator und echten
    MCP-Servern."""
import asyncio
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "projekte" / "personal_agent"))
sys.path.insert(0, str(ROOT / "projekte" / "agent_ollama"))
sys.path.insert(0, str(Path(__file__).resolve().parent))

import ollama_client  # noqa: E402
from kern import PersonalAgent, standard_server  # noqa: E402
from ollama_emulator import Emulator, text, werkzeug_aufruf  # noqa: E402


def lauf(skripte, basis, freigabe):
    emu = Emulator(skripte)
    ollama_client.BASIS = emu.url
    protokoll = basis / "protokoll.jsonl"
    async def main():
        async with PersonalAgent("test", standard_server(basis),
    freigabe, protokoll) as a:
```

```
            return await a.frage("Aufgabe")
    try:
        antwort = asyncio.run(main())
    finally:
        emu.stoppen()
    return antwort, emu, protokoll


def test_alle_server_werden_gestartet_und_werkzeuge_gesammelt(tmp_path):
    (tmp_path / "arbeit").mkdir()
    emu = Emulator([text("fertig")])
    ollama_client.BASIS = emu.url
    async def main():
        async with PersonalAgent("test", standard_server(tmp_path),
    lambda w, a: False) as a:
            await a.frage("Hallo")
            return set(a.werkzeuge)
    try:
        namen = asyncio.run(main())
    finally:
        emu.stoppen()
    for erwartet in ["systeminfo", "speicher", "lese_datei", "ping",
    "abfrage", "merke", "finde"]:
        assert erwartet in namen, erwartet
    werkzeuge = [w["function"]["name"] for w in emu.anfragen[0]["tools"]]
    assert "merke" in werkzeuge and len(werkzeuge) == len(namen)


def test_lesewerkzeug_laeuft_ohne_freigabe(tmp_path):
    (tmp_path / "arbeit").mkdir()
    antwort, emu, protokoll = lauf(
        [werkzeug_aufruf("speicher"), text("Speicher geprüft.")],
    tmp_path, lambda w, a: False)
    assert antwort == "Speicher geprüft."
    eintrag = json.loads(protokoll.read_text(encoding="utf-8").splitlines(
    )[0])
    assert eintrag["werkzeug"] == "speicher" and eintrag["erlaubt"] is True


def test_schreibaktion_ohne_freigabe_wird_abgelehnt(tmp_path):
    (tmp_path / "arbeit").mkdir()
    antwort, emu, protokoll = lauf(
        [werkzeug_aufruf("merke", {"schluessel": "x", "text": "Geheim"}),
         text("Ich habe nichts gespeichert.")], tmp_path, lambda w, a:
    False)
    meldung = [m for m in emu.anfragen[1]["messages"] if m["role"] ==
    "tool"][0]["content"]
    assert "Abgelehnt" in meldung
    assert not (tmp_path / "gedaechtnis.db").exists() or "Geheim" not in \
        (tmp_path / "gedaechtnis.db").read_bytes().decode("latin-1")


def test_freigegebene_notiz_wird_gespeichert_und_wiedergefunden(tmp_path):
    (tmp_path / "arbeit").mkdir()
    antwort, emu, protokoll = lauf(
```

```
    [werkzeug_aufruf("merke", {"schluessel": "tee", "text": "Der
Nutzer mag grünen Tee"}),
     werkzeug_aufruf("finde", {"suchtext": "grünen Tee"}),
     text("Du magst grünen Tee.")], tmp_path, lambda w, a: True)
assert antwort == "Du magst grünen Tee."
fund = [m for m in emu.anfragen[2]["messages"] if m["role"] ==
"tool"][-1]["content"]
assert "tee" in fund
zeilen = protokoll.read_text(encoding="utf-8").splitlines()
assert len(zeilen) == 2
```

Dienst-Datei für den Betrieb unter Debian:

Datei: betrieb/personal-agent.service

```
[Unit]
Description=PERSONAL AI AGENT (MCP Edition)
After=network-online.target

[Service]
Type=simple
User=agent
WorkingDirectory=/opt/personal-agent
Environment=PAGENT_BASIS=/var/lib/personal-agent
Environment=OLLAMA_MODELL=llama3.2
ExecStart=/opt/personal-agent/.venv/bin/python -m cli
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/personal-agent
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Etappe 1: LLM-Anbindung. Das Modul `ollama_client.py` aus Kapitel 7 wird übernommen. Der Test mit dem Emulator prüft die Anfrage und die Antwort.

Etappe 2: MCP-Client. Der Agent startet alle Server über stdio. Jeder Server erhält nur die Umgebungsvariablen, die er braucht. Die Tests prüfen, dass alle Werkzeuge gesammelt werden.

Etappe 3: MCP-Server. Die Server selbst sind in den Kapiteln 6 bis 11 beschrieben. Der Abschlussprojekt-Test prüft, dass sie zusammen funktionieren.

Etappe 4: Agentenlogik. Die Schleife sendet die Nachrichten, führt die Aufrufe aus und gibt die Ergebnisse zurück. Die Schrittgrenze liegt bei sechs Schritten.

Etappe 5: Gedächtnis. Notizen werden über den Gedächtnis-Server gespeichert und wiedergefunden. Das Modell entscheidet, wann es sich etwas merkt. Die Richtlinie verlangt dafür eine Zustimmung.

Etappe 6: Benutzeroberfläche. Das Terminal zeigt die Antworten und fragt bei schreibenden Aktionen nach. Die Oberfläche ist bewusst einfach. Eine grafische Oberfläche ist möglich, berührt aber die Freigabe nicht.

Etappe 7: Sicherheit. Die Rolle des Agenten ist `bearbeiter`. Damit darf er lesen und schreiben, aber nicht administrieren. Schreibaktionen verlangen eine Freigabe. Die Pfadgrenze, die API-Hostliste und die Bestätigung für Löschungen kommen aus den Projekten B, D und E.

Etappe 8: Tests. Vier Ende-zu-Ende-Tests prüfen den Ablauf: alle Server starten, Lesen ohne Freigabe, Schreiben ohne Freigabe wird abgelehnt, Schreiben mit Freigabe wird gespeichert und wiedergefunden. Ausgabe aus dem Testlauf: `4 passed`.

Etappe 9: Protokoll. Jede Entscheidung der Richtlinie landet in `protokoll.jsonl`. Der Test prüft die Anzahl der Einträge.

Etappe 10: Konfiguration. Die Umgebungsvariablen `PAGENT_BASIS`, `OLLAMA_MODELL` und `OLLAMA_URL` steuern den Betrieb.

Etappe 11: Betrieb als Dienst. Die Datei `personal-agent.service` startet den Agenten unter einem eigenen Benutzer. Sie wird mit `systemd-analyze verify` auf Syntax geprüft. Der Pfad zum Programm muss auf dem Zielsystem existieren.

Etappen 12 bis 16: Dokumentation, Deployment-Strategie, Protokollauswertung, Wiederherstellungsprobe und Abnahme. Sie folgen dem Vorgehen aus Kapitel 14.

15.6 Grenzen des Abschlussprojekts

Der Agent wurde mit einem Emulator getestet. Die Qualität der Antworten eines echten Modells ist nicht geprüft. Das Verhalten bei langen Gesprächen ist nicht untersucht. Die Windows-Teile aus Kapitel 9 sind nicht im Projekt enthalten. Der Betrieb in einem Container ist nicht geprüft.

Quellen zu Kapitel 15: Die Quellen der Projekte A bis G und der Kapitel 6 bis 14 gelten auch hier.

Kapitel 16: Fehleranalyse und Troubleshooting

Die folgende Tabelle nennt Fehler, die bei der Entwicklung dieses Buches tatsächlich aufgetreten sind, und die jeweilige Lösung.

Fehlerbild	Ursache	Lösung
Server findet Datei nicht, obwohl sie existiert	Die Umgebungsvariable wird nicht an den Serverprozess weitergegeben, das SDK übernimmt nur eine Auswahl	Variable im Parameter `env` des Serverstarts übergeben
Fehler „unhandled errors in a TaskGroup“	Eine Ausnahme im Client-Block wird in eine ExceptionGroup verpackt	Ausnahme nach dem Verlassen des Blocks auslösen
Ablehnung erscheint als Absturz im Server	Erwartete Ablehnung als normale Ausnahme ausgelöst	`ToolError` verwenden
Protokollverbindung bricht ab, unklare Zeichen im Datenstrom	Der Server schreibt mit `print` in die Standardausgabe	Protokollmeldungen über `logging` an die Fehlerausgabe
`ping` fehlt, Werkzeug meldet Fehler	Das Programm ist auf dem System nicht installiert	Fehlermeldung ist beabsichtigt, andere Werkzeuge bleiben nutzbar
„attempt to write a readonly database“	Schreibanweisung über die Nur-Lese-Verbindung	Schreiben über das Werkzeug mit Bestätigung
Testlauf schlägt fehl, obwohl der Code stimmt	Testdaten aus einem früheren Lauf liegen noch im Verzeichnis	Testverzeichnis vor jedem Lauf löschen
Modell ruft kein Werkzeug auf	Das Modell unterstützt Werkzeugaufrufe nicht zuverlässig	Modell wechseln, Werkzeugbeschreibung präzisieren
Agent liefert keine Antwort, bricht ab	Schrittgrenze erreicht, das Modell wiederholt denselben Aufruf	Aufgabe kleiner formulieren, Werkzeugausgabe prüfen

Fehlerbild	Ursache	Lösung
Schreibaktion wird nicht ausgeführt	Keine Freigabe erteilt	Erwartetes Verhalten, Freigabe prüfen
Ältere Clients verbinden sich nicht	Revision 2025-11-25 mit dem Handshake	SDK in Version 2.x nutzen, es bedient beide Revisionen
Werkzeugergebnis fehlt im Modell	Ergebnis nicht als Nachricht mit der Rolle `tool` zurückgegeben	Schleife prüfen, Nachricht anhängen

Protokollanalyse

Die Protokolldatei enthält eine Zeile pro Entscheidung. Ein Blick auf die Ablehnungen zeigt, ob der Agent einen Fehler wiederholt. Ein Befehl wie `grep erlaubt\":false protokoll.jsonl` findet sie. Der JSON-Eintrag nennt den Grund, und der zeigt, ob die Rolle, das Werkzeug oder die Freigabe die Ursache ist.

Quellen zu Kapitel 16: Die Tabelle beruht auf den Testläufen dieses Buches. Die Fehler sind dort dokumentiert und reproduzierbar.

Kapitel 17: Die Zukunft autonomer KI-Agenten

17.1 Agentische KI

Agentische KI bezeichnet Systeme, die Ziele verfolgen, Werkzeuge nutzen und ihre Schritte selbst planen. Der Begriff ist breit und wird unterschiedlich verwendet. Das Buch versteht darunter Systeme, die innerhalb von Grenzen handeln, die ein Mensch festlegt.

17.2 Lokale und hybride Agenten

Lokale Agenten laufen vollständig auf eigener Hardware, wie das Abschlussprojekt. Hybride Agenten nutzen lokale Modelle für vertrauliche Aufgaben und entfernte Dienste für rechenintensive. Die Wahl richtet sich nach Datenschutz, Kosten und Leistung. Beide Formen sind heute möglich.

17.3 Das MCP-Ökosystem und Interoperabilität

Das Protokoll verbreitet sich, weil viele Anwendungen und Dienste es sprechen. Ein Server, den man einmal schreibt, funktioniert mit verschiedenen Clients. Die Interoperabilität ist die Stärke des Standards. Sie setzt aber voraus, dass Spezifikation und Umsetzung zusammenpassen. Die Revision 2026-07-28 hat das Protokoll verändert, und ältere Umsetzungen brauchen Anpassungen.

17.4 Agenten als digitale Mitarbeitende

Ein Agent kann Aufgaben übernehmen, die viel Routinearbeit enthalten: Auswertungen, Prüfungen, Recherchen. Die Verantwortung bleibt beim Menschen. Ein Agent ist ein Werkzeug mit eigenen Fehlern, nicht ein Kollege, der für seine Arbeit einsteht.

17.5 Chancen und Risiken

Die Chancen liegen in Zeitersparnis, Konsistenz und Zugang zu Daten. Die Risiken liegen in Fehlern, Manipulation, Datenabfluss und zu weit gefassten Rechten. Die Maßnahmen aus Kapitel 13 und Band 8 begrenzen sie, beseitigen sie aber nicht.

17.6 Verfügbares und Zukünftiges

Verfügbar sind heute: die Spezifikation 2026-07-28, das SDK in Version 2.x, lokale Modelle mit Werkzeugaufrufen, die in diesem Buch beschriebenen Server und das Abschlussprojekt.

Nicht verfügbar oder nicht geprüft sind: Erweiterungen wie Tasks und MCP Apps in produktiver Form, die Vektorsuche im Gedächtnis als getesteter Bestandteil, verlässliche Autorisierung für entfernte Server im Heimnetz ohne eigene Einrichtung, und die Windows-Anbindung in der Referenzumgebung.

Die Entwicklung ist schnell, die Spezifikation ändert sich. Dieses Buch beschreibt den Stand vom 10. November 2026.

Quellen zu Kapitel 17:

Model Context Protocol, Spezifikation Revision 2026-07-28 und offizielle Ankündigungen, modelcontextprotocol.io.

MCP Python SDK, Version 2.x, github.com/modelcontextprotocol/python-sdk.

Glossar

Agent. System, in dem ein Sprachmodell in einer Schleife Werkzeuge aufruft, bis eine Aufgabe erledigt ist.

Agent Loop. Die Schleife eines Agenten aus Modellaufruf, Werkzeugausführung und Ergebnisrückgabe.

Client. Bestandteil des Hosts, der eine Verbindung zu einem MCP-Server hält.

Embedding. Vektor aus Zahlen, der die Bedeutung eines Textes beschreibt.

Freigabe. Zustimmung des Benutzers vor einer schreibenden oder administrativen Aktion.

Host. Anwendung, mit der der Benutzer arbeitet und die MCP-Clients enthält.

JSON-RPC. Format der Nachrichten zwischen Client und Server in MCP.

Least Privilege. Grundsatz der minimalen Rechte.

MCP. Model Context Protocol, offener Standard für die Anbindung von Werkzeugen und Daten an KI-Anwendungen.

Prompt (MCP). Vorlage, die ein Server bereitstellt und mit Parametern füllt.

Resource. Datenquelle, die ein Server bereitstellt und die ein Client lesen kann.

Server (MCP). Programm, das Werkzeuge, Ressourcen und Prompts bereitstellt.

stdio. Transport, bei dem Client und Server über Standardeingabe und -ausgabe eines Prozesses sprechen.

Streamable HTTP. Transport für entfernte Server über HTTP.

Tool. Funktion, die ein Server bereitstellt und der Client aufrufen kann.

Tool Calling. Mechanismus, mit dem ein Modell einen Werkzeugaufruf mit Argumenten verlangt.

ToolError. Ausnahme des SDK für erwartete Ablehnungen, die als Fehlerergebnis zurückgegeben werden.

Zustandslos. Eigenschaft der Revision 2026-07-28: Jede Anfrage trägt ihre Angaben selbst, ohne Sitzung.

Literatur- und Quellenverzeichnis

- Model Context Protocol, Spezifikation Revision 2026-07-28, modelcontextprotocol.io. Änderungsprotokoll und Abschnitte zu Architektur, Nachrichten, Transporten, Tools, Resources, Prompts und Sicherheit.
- Model Context Protocol, Ankündigungen „The 2026-07-28 MCP Specification Release Candidate“ und „The 2026-07-28 Specification“, blog.modelcontextprotocol.io.
- MCP Python SDK, Version 2.0.0 vom 28. Juli 2026 und Version 2.3.0, github.com/modelcontextprotocol/python-sdk.
- Paket mcp, PyPI, pypi.org/project/mcp.
- JSON-RPC 2.0 Specification, jsonrpc.org.
- Ollama, Dokumentation zur Chat-Schnittstelle und zu Werkzeugen, ollama.com.
- Python-Dokumentation: asyncio, json, logging, pathlib, socket, sqlite3, subprocess, urllib. docs.python.org.
- SQLite-Dokumentation, sqlite.org/docs.html.
- systemd-Dokumentation, freedesktop.org.
- Linux-Kernel-Dokumentation zu procfs, kernel.org.
- Docker-Dokumentation, docs.docker.com.
- Pytest-Dokumentation, docs.pytest.org.
- Band 1 bis Band 9 der Reihe KI PRAXIS.

Stichwortverzeichnis

Agent Loop, Kapitel 2, 8. Authentifizierung, Kapitel 13. Berechtigungen, Kapitel 6, 13. Client, Kapitel 3, 8. Container, Kapitel 14. Datenbank, Kapitel 10. Embedding, Kapitel 11. Freigabe, Kapitel 8, 13. Gedächtnis, Kapitel 11. Host, Kapitel 3. JSON-RPC, Kapitel 3. Least Privilege, Kapitel 13. Multi-Agent, Kapitel 12. Ollama, Kapitel 7. Prompt, Kapitel 4. Prompt Injection, Kapitel 13. Protokollierung, Kapitel 13, 14. Resource, Kapitel 4. Server, Kapitel 3, 6. stdio, Kapitel 3. Streamable HTTP, Kapitel 3, 13. systemd, Kapitel 14. Tool, Kapitel 4, 6. ToolError, Kapitel 4, 6. Troubleshooting, Kapitel 16. Zustandslos, Kapitel 3.

KI PRAXIS

Band 10 führt in das Model Context Protocol ein und zeigt, wie man eigene MCP-Server, einen Client und einen Agenten mit lokalem Sprachmodell entwickelt. Die Projekte reichen von Systeminformationen über Datei-, Netzwerk- und Datenbankzugriff bis zu einem Gedächtnis, einem Multi-Agent-System und einem Berechtigungsmodell.

Das Abschlussprojekt PERSONAL AI AGENT verbindet alle Bausteine. Freigaben, Protokolle und Schutzregeln begrenzen, was der Agent tun darf.

Die weiteren Bände der Reihe KI PRAXIS: Band 1 bis Band 9 behandeln Alltag, Recherche, Automatisierung, lokale KI, Datenanalyse, Dokumente, Lernen, Sicherheit und Recht.

Eugen Becker | Stand 10. November 2026